



Universidad Privada del Valle
Facultad de Informática y Electrónica
Carrera de Licenciatura en Ingeniería de Sistemas

**MÓDULO DE INTELIGENCIA PREDICTIVA DEL ERP ISI MUSTANG
BASADO EN EL APROVECHAMIENTO DE DATOS HISTÓRICOS
OPERATIVOS PARA ANTICIPAR DESVIACIONES Y RIESGOS EN
LA GESTIÓN DE PROYECTOS DE INGENIERÍA EPC**

**PROYECTO DE GRADO PARA OPTAR
AL TÍTULO DE
LICENCIATURA EN INGENIERÍA DE SISTEMAS**

POSTULANTE: FRANCO JAVIER TORREZ ALVARADO
TUTOR: ROLANDO LARA SANCHEZ

Santa Cruz – Bolivia

2026

Índice

INTRODUCCIÓN	1
CAPÍTULO I: MARCO GENERAL	2
1.1 ANTECEDENTES	3
1.2 PLANTEAMIENTO DEL PROBLEMA	5
1.2.1 FORMULACIÓN DEL PROBLEMA	9
1.3 OBJETIVOS	9
1.3.1 OBJETIVO GENERAL	10
1.3.2 OBJETIVOS ESPECÍFICOS	10
1.4 JUSTIFICACIÓN	10
1.4.1 JUSTIFICACIÓN TÉCNICA	11
1.4.2 JUSTIFICACIÓN ECONÓMICA	12
1.4.3 JUSTIFICACIÓN SOCIAL	12
1.5 METODOLOGÍA	13
1.5.1 TIPO DE INVESTIGACIÓN	13
1.5.2 MÉTODO DE INVESTIGACIÓN	13
1.5.3 TÉCNICAS E INSTRUMENTOS DE INVESTIGACIÓN	14
1.5.4 POBLACIÓN Y MUESTRA	15
1.6 DELIMITACIÓN DE LA INVESTIGACIÓN	16
1.6.1 DELIMITACIÓN TEMÁTICA	16
1.6.2 DELIMITACIÓN ESPACIAL	16
1.6.3 DELIMITACIÓN TEMPORAL	17
CAPÍTULO II: MARCO TEÓRICO Y TECNOLÓGICO	18
2.1 SISTEMAS ERP	19
2.1.1 DEFINICIÓN Y CARACTERÍSTICAS	19
2.1.2 ERP EN EMPRESAS DE INGENIERÍA	20
2.1.3 ERP EN EL SECTOR DE PETRÓLEO, GAS Y MINERÍA	22
2.2 APRENDIZAJE AUTOMÁTICO	23
2.2.1 TIPOS DE APRENDIZAJE	23
2.2.2 APRENDIZAJE SUPERVISADO	24
2.2.3 ALGORITMOS: GRADIENT BOOSTING Y XGBOOST	28
2.2.4 FEATURE ENGINEERING	30
2.3 INGENIERÍA DE DATOS	31
2.3.1 PROCESOS ETL Y ELT	31
2.3.2 ARQUITECTURA MEDALLION (BRONZE, SILVER, GOLD)	33
2.3.3 CALIDAD DE DATOS	35
2.4 ANALÍTICA PREDICTIVA	37
2.4.1 DEFINICIÓN Y APLICACIONES	37
2.4.2 ANALÍTICA PREDICTIVA EN GESTIÓN DE PROYECTOS	39

2.4.3	PREDICCIÓN DE DESVIACIONES Y COSTOS	40
2.5	GESTIÓN DE PROYECTOS EPC	43
2.5.1	DEFINICIÓN Y CARACTERÍSTICAS	43
2.5.2	CERTIFICACIONES Y CONTROL DE AVANCE	44
2.5.3	DESVIACIONES PRESUPUESTARIAS Y DE CRONOGRAMA	46
2.6	INGENIERÍA DE SOFTWARE	48
2.6.1	PROCESO DE DESARROLLO DE SOFTWARE	48
2.6.2	METODOLOGÍA DE DESARROLLO ITERATIVA E INCREMENTAL	50
2.6.3	ARQUITECTURA DE SOFTWARE POR CAPAS	52
2.7	TECNOLOGÍAS DEL STACK DE DESARROLLO	53
2.7.1	PYTHON Y SU ECOSISTEMA ML	54
2.7.2	DUCKDB Y POLARS	55
2.7.3	FASTAPI COMO MICROSERVICIO	56
2.7.4	NESTJS Y ANGULAR	57
2.8	GESTIÓN DE IDENTIDAD Y ACCESO	60
2.8.1	OAuth 2.0 Y OPENID CONNECT	60
2.8.2	KEYCLOAK	63
2.9	MODEL CONTEXT PROTOCOL (MCP)	65
2.9.1	DEFINICIÓN Y ARQUITECTURA	65
2.9.2	INTEGRACIÓN DE SISTEMAS EMPRESARIALES CON MODELOS DE LENGUAJE	68
2.10	INTELIGENCIA DE NEGOCIOS	69
2.10.1	DEFINICIÓN Y REPORTE DESCRIPTIVOS	69
2.10.2	POWER BI	71
CAPÍTULO III: ANÁLISIS Y DISEÑO DEL SISTEMA PROPUESTO		74
3.1	ANÁLISIS DE DATOS HISTÓRICOS	75
3.1.1	EXPLORACIÓN Y CARACTERIZACIÓN DE LAS FUENTES DE DATOS	75
3.1.2	IDENTIFICACIÓN DE VARIABLES PREDICTORAS	75
3.1.3	DEFINICIÓN DE CASOS DE USO PREDICTIVOS	75
3.2	ANÁLISIS DE PROCESOS DEL PORTAL ANTERIOR	75
3.2.1	RELEVAMIENTO DE PROCESOS DEL PORTAL ANTERIOR	75
3.2.2	PROCESOS A REIMPLEMENTAR EN EL ERP	75
3.2.3	DEFECTOS Y BRECHAS QUE AFECTAN LA CALIDAD DE LOS DATOS	75
3.3	REQUERIMIENTOS DEL SISTEMA	75
3.3.1	REQUERIMIENTOS FUNCIONALES	75
3.3.2	REQUERIMIENTOS NO FUNCIONALES	75
3.4	DISEÑO DE LA ARQUITECTURA	76
3.4.1	ARQUITECTURA POR CAPAS	76
3.4.2	DISEÑO DEL PIPELINE DE DATOS	76
3.4.3	DISEÑO DE MODELOS ML	76
3.4.4	DISEÑO DE LA API PREDICTIVA	76

3.4.5	DISEÑO DEL DASHBOARD	76
3.4.6	DISEÑO DE LA AUTENTICACIÓN CENTRALIZADA	76
3.4.7	DISEÑO DE LAS INTEGRACIONES CON MCP Y POWER BI	76
CAPÍTULO IV: CONSTRUCCIÓN E IMPLEMENTACIÓN DEL SISTEMA		77
4.1	INCREMENTO 1: PROCESOS DEL PORTAL ANTERIOR MIGRADOS AL ERP . . .	78
4.2	INCREMENTO 2: AUTENTICACIÓN CENTRALIZADA CON KEYCLOAK	78
4.3	INCREMENTO 3: PIPELINE DE DATOS Y ENTRENAMIENTO DE MODELOS . . .	78
4.3.1	INTEGRACIÓN DUCKDB CON MYSQL Y POSTGRESQL	78
4.3.2	TRANSFORMACIÓN DE DATOS CON POLARS Y FEATURE ENGINEERING	78
4.3.3	MODELO DE DESVIACIÓN EN CERTIFICACIONES	78
4.3.4	MODELO DE SOBRECOSTO DE PROYECTOS	78
4.3.5	MODELO DE RETRASO EN LA FECHA DE CIERRE	78
4.4	INCREMENTO 4: MICROSERVICIO FASTAPI E INTEGRACIÓN CON EL ERP . . .	78
4.4.1	DESARROLLO DEL MICROSERVICIO FASTAPI	78
4.4.2	ENDPOINTS DE PREDICCIÓN EN NESTJS	78
4.4.3	DASHBOARD DE INTELIGENCIA PREDICTIVA EN ANGULAR	79
4.5	INCREMENTO 5: INTEGRACIÓN CON POWER BI Y MCP	79
4.5.1	API DE DATOS PARA POWER BI	79
4.5.2	SERVIDOR MCP	79
4.6	EVALUACIÓN Y VALIDACIÓN	79
4.6.1	MÉTRICAS DE RENDIMIENTO DE MODELOS	79
4.6.2	PRUEBAS CON USUARIOS CLAVE	79
CONCLUSIONES		80
RECOMENDACIONES		81
REFERENCIAS BIBLIOGRÁFICAS		82

ÍNDICE DE FIGURAS

1.1	Diagrama de Ishikawa: causas de la gestión reactiva de riesgos en los proyectos EPC de ISI Mustang	7
1.2	Árbol del problema: causas y efectos de la gestión reactiva de desviaciones y riesgos en los proyectos de ISI Mustang	9

ÍNDICE DE CUADROS

1.1	Análisis FODA de ISI Mustang Bolivia respecto a la explotación predictiva de los datos de su ERP	8
2.1	Métricas de evaluación de modelos supervisados y su uso en el proyecto	27
2.2	Capas de la arquitectura <i>medallion</i> y su correspondencia en el proyecto	35
2.3	Dimensiones de calidad de datos adoptadas en el proyecto	36
2.4	Estudios revisados sobre predicción de desviaciones, costos y plazos en proyectos	42
2.5	Variaciones, índices y pronóstico de la gestión del valor ganado	45
2.6	Modelos de proceso de software comparados	50
2.7	Tecnologías del sistema por capa de la arquitectura	59
2.8	Roles y flujos de OAuth 2.0 / OpenID Connect y su uso en el proyecto	63
2.9	Primitivas del servidor MCP y su uso previsto en el proyecto	67
2.10	Herramientas de BI consideradas y su forma de acceso a los datos del ERP	73

INTRODUCCIÓN

CAPÍTULO I

MARCO GENERAL

1.1 ANTECEDENTES

Los sistemas de planificación de recursos empresariales (ERP) constituyen el núcleo operativo de las empresas de ingeniería: registran de forma continua los datos de proyectos, certificaciones, recursos humanos, horas trabajadas, requisiciones y compras que sostienen la operación diaria. Sin embargo, como señalan Jawad y Villányi (2024), la mayoría de las organizaciones utilizan estos sistemas principalmente para la gestión operativa y la generación de reportes descriptivos, sin explotar el potencial predictivo de los datos históricos que acumulan. En el ámbito de la gestión de proyectos de ingeniería, el aprendizaje automático ha demostrado superar en precisión a los métodos tradicionales de planificación y control: Shamim et al. (2025) concluyen, a partir de una revisión sistemática de 39 estudios publicados entre 2016 y 2024, que los modelos de inteligencia artificial mejoran significativamente la estimación de costos frente a los métodos convencionales, mientras que Ottaviani et al. (2025) proponen pipelines automatizados capaces de predecir costos y duración de proyectos con mayor exactitud que las herramientas basadas en la gestión del valor ganado (*Earned Value Management*).

En cuanto a trabajos que abordan problemas análogos al planteado, Nguyen et al. (2026) desarrollaron un modelo de XGBoost explicable para predecir sobrecostos en proyectos de construcción a partir de datos históricos estructurados, superando en precisión a modelos tradicionales como la regresión lineal y los árboles de decisión simples; no obstante, su estudio se sustenta en conjuntos de datos del sector construcción y no contempla la integración del modelo en el sistema de información operativo de una empresa. İnan et al. (2022) reportan que los modelos de aprendizaje automático entrenados sobre datos históricos de proyectos producen estimaciones de costo más precisas que los índices tradicionales del valor ganado. Por su parte, Ottaviani et al. (2025) evidencian que la integración de múltiples fuentes de datos mejora la precisión predictiva; aun así, su pipeline opera sobre datos consolidados de diversas organizaciones y no sobre el historial operativo de una empresa en particular. En el terreno de los sistemas ERP del sector, Al Jafari y Nair (2018) analizan, mediante un estudio de caso con enfoque mixto en el Sultanato de Omán, cómo la implementación del ERP influye en la eficiencia y la productividad de las empresas de petróleo y gas, y concluyen recomendando su adopción; su trabajo se ocupa del ERP como registro de la operación y no de la explotación predictiva de los datos que acumula.

Los antecedentes revisados confirman la viabilidad técnica de aplicar aprendizaje automático para anticipar desviaciones y riesgos en proyectos de ingeniería, pero comparten una limitación común: se desarrollan sobre conjuntos de datos públicos o multiorganizacionales y no se integran en el sistema de gestión operativo de la empresa que origina los datos. Tampoco se identificaron antecedentes de aplicación de inteligencia predictiva sobre los datos del ERP de ISI Mustang ni trabajos similares en el contexto de empresas de ingeniería bolivianas. Esta doble brecha, la ausencia de soluciones integradas al ERP y la falta de precedentes en el contexto local, constituye el punto de partida del presente proyecto de grado.

ISI Mustang Bolivia es una empresa con más de 25 años de operación en ingeniería y servicios tecnológicos para los sectores de gas y petróleo, energía y minería, con sede en Santa Cruz de la Sierra. Ejecuta proyectos integrales de tipo EPC (*Engineering, Procurement and Construction*), modalidad en la que asume de forma completa el diseño de ingeniería, la procura de equipos y materiales, y la construcción de las instalaciones. Para planificar y controlar estos proyectos, la empresa se ha apoyado durante más de una década en un sistema de información propio, el Portal ISIMustang Bolivia, desarrollado en PHP sobre una base de datos MySQL 5.7 cuyo esquema comprende 175 tablas. En ese sistema se encuentra el histórico de más de diez años de operación: proyectos y centros de costo, actividades, facturación, horas trabajadas, requisiciones y compras. De forma complementaria, parte del control de proyectos se lleva en planillas Excel fuera del sistema.

En la actualidad la empresa reconstruye ese portal como un ERP nuevo, de desarrollo propio y en curso, con un backend en NestJS y TypeScript sobre PostgreSQL y un frontend en Angular. El ERP nuevo reimplementa progresivamente los módulos del portal anterior y es el sistema sobre el que operan los proyectos activos; el portal anterior, por su parte, conserva el histórico que no ha sido migrado. Esta coexistencia de dos sistemas, uno con el pasado y otro con el presente de la operación, define el escenario técnico del presente trabajo: los datos para aprender están en un esquema y los proyectos sobre los que se quiere anticipar riesgos en otro.

Se parte de la hipótesis de que ese histórico contiene un activo de especial valor para el aprendizaje automático: el registro del avance facturado de cada proyecto frente a lo planificado, junto con las horas, compras y presupuestos asociados, que permitiría reconstruir las desviaciones

ocurridas y, en la medida en que las causas hayan sido registradas de forma consistente, conformar un conjunto de datos etiquetado para aprendizaje supervisado. La confirmación de esa hipótesis corresponde al análisis de datos del Capítulo III. Nguyen et al. (2026) demuestran que modelos como XGBoost alcanzan alta precisión en la predicción de sobrecostos cuando se dispone de datos históricos estructurados, lo que respalda la viabilidad de aprovechar el historial de ISI Mustang para anticipar desviaciones y riesgos operativos. El presente trabajo propone, en consecuencia, el desarrollo de un módulo de inteligencia predictiva integrado al ERP de ISI Mustang que, mediante técnicas de aprendizaje automático aplicadas sobre ese histórico, permita anticipar desviaciones y riesgos en la gestión de sus proyectos EPC, en el marco de una modernización del ERP que incorpora autenticación centralizada, integración con herramientas de inteligencia de negocios y exposición de sus datos mediante el protocolo MCP.

1.2 PLANTEAMIENTO DEL PROBLEMA

Los proyectos EPC que ejecuta ISI Mustang para los sectores de gas, petróleo, minería y energía involucran alta complejidad técnica, presupuestos considerables y plazos estrictos. Para planificarlos y controlarlos, avances, certificaciones, recursos, compras y horas trabajadas, la empresa se apoya en su sistema de información, que acumula más de diez años de datos históricos operativos. Sin embargo, dicha información se utiliza únicamente con fines descriptivos y de control inmediato: sirve para saber qué ocurrió y cuánto se ejecutó, pero no existe un mecanismo que permita identificar patrones en el historial ni anticipar comportamientos futuros. Como consecuencia, la detección de desviaciones presupuestarias, retrasos y cuellos de botella ocurre de forma reactiva, una vez que la desviación ya se materializó y su corrección resulta más costosa, lo que limita la capacidad de gestión proactiva de riesgos en proyectos de alta envergadura. La literatura del sector confirma que se trata de un problema recurrente: Habibi et al. (2018), a partir de una revisión de más de doscientos artículos sobre proyectos EPC, señalan que más de la mitad de los proyectos enfrenta retrasos significativos y escaladas de costo importantes, identifican el cambio de diseño como la causa más común de retraso y sobrecosto, seguida de la escasez de recursos y la fluctuación de precios, y observan que todo retraso termina traducéndose en sobrecosto.

A la gestión reactiva se suma la situación del propio sistema de información. El histórico reside en el portal anterior, cuyos registros están dispersos en módulos sin integración analítica y cuya calidad depende de la disciplina de carga de cada usuario; el ERP nuevo, en construcción, todavía no cubre todos los procesos del portal anterior y presenta defectos que ensucian los datos que registra; y ninguno de los dos cuenta con un módulo de analítica predictiva, una capa de visualización de predicciones ni un mecanismo de alertas tempranas basado en datos. Anticipar riesgos exige, por tanto, no solo construir modelos, sino también preparar el ERP para que sus datos y sus integraciones lo permitan.

El diagrama de Ishikawa de la Figura 1.1 sintetiza las causas de esta problemática agrupadas en cuatro categorías. En la categoría de *procesos*, la detección de desviaciones solo cuando ya ocurrieron, la toma de decisiones sin retroalimentación histórica y la ausencia de métricas de anticipación de riesgo. En la de *datos*, los registros dispersos en módulos sin integración analítica, la inexistencia de variables predictoras definidas y un historial de diez años sin estructura para el aprendizaje automático. En la de *modelos*, la ausencia de un pipeline de entrenamiento y validación, las estimaciones basadas únicamente en criterio y experiencia, y la inexistencia de modelos entrenados sobre datos operativos. Finalmente, en la de *sistema*, un ERP sin módulo de analítica predictiva, sin alertas tempranas basadas en datos y sin una capa de visualización de predicciones. La figura recoge las causas relevadas en la etapa de perfil; el análisis posterior de los dos sistemas de información añadió, en la categoría de datos, la coexistencia del histórico y de los proyectos activos en esquemas distintos y los defectos de registro del ERP nuevo, causa que se incorpora al árbol del problema de la Figura 1.2.

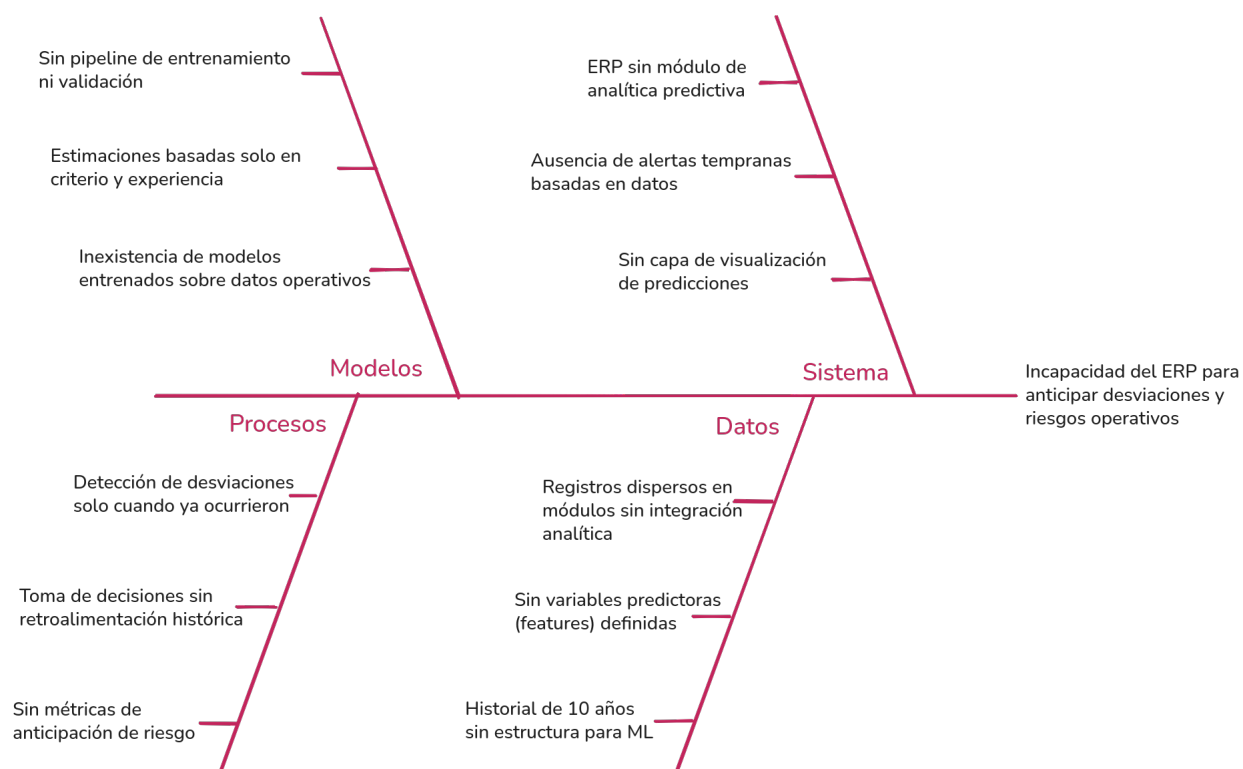


Figura 1.1. Diagrama de Ishikawa: causas de la gestión reactiva de riesgos en los proyectos EPC de ISI Mustang

Fuente: Elaboración propia, 2026.

El análisis FODA del Cuadro 1.1 resume la situación de la empresa frente a la explotación predictiva de sus datos. Entre las fortalezas destacan el volumen del histórico acumulado y el control total sobre el sistema de información, al tratarse de un desarrollo propio; entre las debilidades, las posibles inconsistencias del registro histórico y la dependencia de la disciplina de carga. Las oportunidades provienen de la demanda de gestión proactiva en el sector energético y de la madurez de las técnicas de análisis predictivo, mientras que las amenazas se concentran en la adopción por parte de los usuarios y en la vigencia de los modelos ante cambios en los procesos.

Cuadro 1.1. Análisis FODA de ISI Mustang Bolivia respecto a la explotación predictiva de los datos de su ERP

Fortalezas • Más de diez años de datos históricos operativos acumulados en el portal anterior. • Registro del avance facturado frente a lo planificado que, de confirmarse en el análisis de datos, permitiría reconstruir las desviaciones ocurridas. • Sistema de información propio, con control total sobre el código y los datos operativos.	Oportunidades • Alta demanda de gestión proactiva en proyectos de ingeniería del sector energético. • Madurez y disponibilidad de técnicas de aprendizaje automático para datos tabulares. • Mejora del posicionamiento competitivo en la estimación y adjudicación de contratos.
Debilidades • Posibles inconsistencias en el registro histórico de información. • Histórico y proyectos activos en dos sistemas con esquemas distintos. • Ausencia de una cultura de análisis de datos en la organización. • Dependencia de la disciplina en el registro operativo para la calidad de los resultados.	Amenazas • Resistencia al cambio por parte de los usuarios ante nuevas herramientas de gestión. • Cambios en los procesos operativos que reduzcan la vigencia de los modelos desarrollados. • Baja adopción si los resultados no se comunican de forma clara a los usuarios clave.

Fuente: Elaboración propia, 2026.

El árbol del problema de la Figura 1.2 ordena esta situación en términos de causas y efectos. El problema central es la gestión reactiva de desviaciones y riesgos en los proyectos de ingeniería de ISI Mustang. Sus causas son un historial de diez años sin estructura para el análisis predictivo, la coexistencia del histórico y de los proyectos activos en dos sistemas con esquemas distintos y con defectos de registro en el ERP nuevo, un ERP sin módulo de analítica predictiva, un control de proyectos reactivo y sin seguimiento anticipado, y la inexistencia de modelos entrenados sobre los datos operativos de la empresa. Sus efectos son los sobrecostos por detección tardía de desviaciones, los retrasos de cronograma no anticipados, los cuellos de botella en la asignación de recursos y la pérdida de competitividad en licitaciones, dado que la estimación de nuevas propuestas no se beneficia de lo aprendido en los proyectos anteriores.

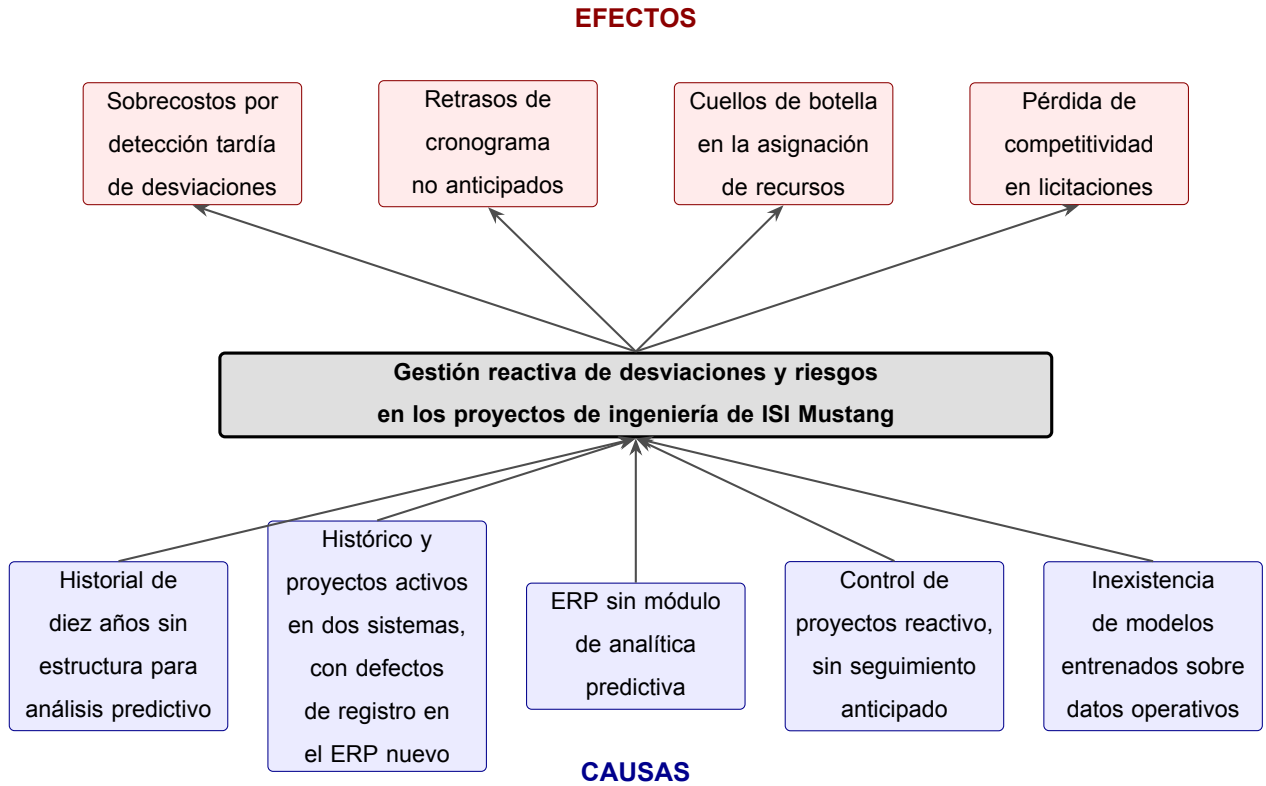


Figura 1.2. Árbol del problema: causas y efectos de la gestión reactiva de desviaciones y riesgos en los proyectos de ISI Mustang

Fuente: Elaboración propia, 2026.

El objeto de estudio del presente trabajo es, en consecuencia, el proceso de gestión y control de desviaciones y riesgos en los proyectos de ingeniería de ISI Mustang, a partir de los datos históricos registrados en su sistema de información.

1.2.1 FORMULACIÓN DEL PROBLEMA

¿Cómo aprovechar los datos históricos del ERP ISI Mustang para anticipar desviaciones y riesgos en los proyectos de ingeniería EPC de la empresa?

1.3 OBJETIVOS

1.3.1 OBJETIVO GENERAL

Desarrollar un módulo de inteligencia predictiva integrado al ERP ISI Mustang, mediante técnicas de aprendizaje automático aplicadas sobre los datos históricos operativos de la empresa, para anticipar desviaciones y riesgos en la gestión de proyectos de ingeniería EPC, en el marco de una modernización del ERP que incorpora autenticación centralizada, integración con herramientas de inteligencia de negocios y exposición de sus datos mediante el protocolo MCP.

1.3.2 OBJETIVOS ESPECÍFICOS

1. Identificar los procesos del portal anterior (Portal ISIMustang Bolivia) pendientes de reimplementar en el ERP y los defectos que afectan la calidad de sus datos operativos.
2. Analizar los datos históricos de proyectos registrados en el sistema legado y en las planillas de control complementarias, para la definición de los casos de uso del módulo predictivo.
3. Diseñar la arquitectura del módulo predictivo y de sus integraciones con el ERP, incluyendo la selección de los modelos de aprendizaje automático adecuados para cada caso de uso.
4. Implementar en el ERP los procesos identificados y la autenticación centralizada con Keycloak (OAuth 2.0 / OpenID Connect), como base para las integraciones del sistema.
5. Construir el pipeline de datos y los modelos predictivos para los casos de uso definidos: desviación en certificaciones, sobrecosto de proyecto y retraso en la fecha de cierre.
6. Integrar al ERP el servicio de predicción con su dashboard de visualización, los reportes descriptivos en Power BI y el servidor MCP.
7. Evaluar la precisión de los modelos y la funcionalidad del sistema mediante métricas de rendimiento y pruebas con usuarios clave de ISI Mustang.

1.4 JUSTIFICACIÓN

1.4.1 JUSTIFICACIÓN TÉCNICA

El sistema de información de ISI Mustang acumula más de diez años de datos operativos distribuidos en múltiples módulos (proyectos y centros de costo, actividades, facturación, horas trabajadas, requisiciones y compras), complementados por las planillas de control que la empresa lleva fuera del sistema. Ese volumen es suficiente para el entrenamiento de modelos de aprendizaje automático sobre datos tabulares. En particular, el portal anterior calculaba indicadores de desvío de costo, de ingresos y de avance por proyecto a partir del avance facturado frente a lo planificado; se parte de la hipótesis de que los datos con los que se calculaban esos indicadores permiten reconstruir las desviaciones ocurridas en cada proyecto y, junto con las causas que los usuarios hayan documentado, conformar un conjunto de datos etiquetado para aprendizaje supervisado. La confirmación de esa hipótesis, tanto de la consistencia del registro de avance facturado frente a lo planificado como de la completitud con que se registraron las causas, corresponde a la exploración de datos que se desarrolla en el Capítulo III; si las causas no resultan aprovechables, la etiqueta se derivaría únicamente de la comparación entre lo planificado y lo ejecutado.

Las técnicas de aprendizaje automático supervisado, en particular los modelos de *gradient boosting* como XGBoost, son suficientemente maduras para aplicarse sobre este tipo de datos, como muestran los antecedentes revisados, y cuentan con implementaciones abiertas y ampliamente utilizadas en el ecosistema de Python. La ingeniería de datos dispone además de herramientas, como DuckDB y Polars, capaces de leer directamente bases MySQL y PostgreSQL y archivos de planilla, y de unificarlos en un pipeline analítico sin infraestructura especializada, lo que resuelve la principal dificultad técnica del caso: que el histórico y los proyectos activos residan en dos sistemas con esquemas distintos.

Por último, la integración del módulo predictivo al ERP es técnicamente viable porque el ERP es un desarrollo propio de la empresa. Ello permite exponer los datos y las predicciones mediante servicios con contratos bien definidos, incorporar la autenticación centralizada con Keycloak como base común de acceso para el ERP, el servicio de predicción, el servidor MCP y Power BI, y corregir en el propio ERP los defectos que degradan la calidad de los datos, algo que no sería posible sobre un paquete comercial cerrado.

1.4.2 JUSTIFICACIÓN ECONÓMICA

Los proyectos de tipo EPC que ejecuta ISI Mustang en los sectores de gas y petróleo, minería y energía involucran presupuestos de alta envergadura, en los que una desviación pequeña en términos porcentuales representa montos significativos. La detección temprana de desviaciones permite actuar cuando las medidas correctivas todavía son baratas, reasignar recursos, adelantar una compra o ajustar el cronograma, y reduce el costo de corrección frente a la detección tardía. La anticipación de cuellos de botella en la asignación de recursos evita sobre costos por sobreasignación o por tiempos improductivos. Asimismo, la capacidad de contrastar nuevas propuestas con el comportamiento de proyectos anteriores mejora la competitividad de la empresa en la estimación y adjudicación de contratos.

La solución se construye, además, sobre activos que la empresa ya posee: sus datos históricos, su ERP y su servidor propio. Las tecnologías del módulo predictivo (Python, DuckDB, Polars, XGBoost, FastAPI) y de la autenticación centralizada (Keycloak) son de código abierto; la única herramienta con licencia comercial es Power BI, cuya adopción para los reportes descriptivos ya está prevista por la empresa con independencia de este trabajo. La inversión se concentra, por tanto, en el desarrollo.

1.4.3 JUSTIFICACIÓN SOCIAL

Una gestión de proyectos más proactiva y basada en datos contribuye a entregas más confiables en sectores críticos para el desarrollo de Bolivia, como los hidrocarburos, la energía y la minería, cuyas instalaciones dependen de que los proyectos de ingeniería se completen en plazo y con la calidad comprometida. La reducción de la gestión reactiva disminuye la carga operativa y las urgencias de último momento de los equipos de proyecto, lo que se traduce en mejores condiciones de trabajo. A nivel más amplio, el proyecto muestra que una empresa boliviana puede capitalizar sus propios datos históricos mediante inteligencia artificial, con herramientas abiertas y sobre infraestructura existente, y deja un antecedente replicable para otras empresas de ingeniería del país.

1.5 METODOLOGÍA

1.5.1 TIPO DE INVESTIGACIÓN

El presente trabajo corresponde a una investigación aplicada, dado que utiliza conocimiento científico existente en el área del aprendizaje automático y de la ingeniería de software para dar solución a un problema concreto en el contexto operativo de ISI Mustang Bolivia. De acuerdo con sus fases, combina tres niveles de alcance: exploratorio, en la etapa inicial de comprensión y caracterización de los datos históricos del portal anterior y de las planillas de control; descriptivo, para identificar y caracterizar las variables operativas relevantes y los patrones de desviación; y correlacional, al modelar las relaciones entre dichas variables, como horas trabajadas, líneas de presupuesto, compras y avance certificado, con el fin de anticipar desviaciones y riesgos en los proyectos.

1.5.2 MÉTODO DE INVESTIGACIÓN

El trabajo adopta un enfoque mixto, dado que combina la recolección y el análisis de datos cuantitativos, los registros históricos del sistema de información, con datos cualitativos provenientes de las entrevistas a usuarios clave. Para operacionalizar este enfoque se emplean los siguientes métodos.

Análisis documental. Se aplica a la revisión de la literatura científica sobre aprendizaje automático, analítica predictiva, ingeniería de datos, sistemas ERP, gestión de identidad y protocolos de integración, con el fin de establecer los fundamentos teóricos del trabajo, y a la documentación del portal anterior y del ERP nuevo, para identificar los procesos pendientes de reimplementar y los defectos que afectan a los datos.

Histórico-lógico. Se aplica sobre los datos acumulados durante más de diez años de operación, con el fin de identificar patrones y tendencias en el comportamiento de los proyectos a lo largo del tiempo y de reconstruir, a partir de ellos, las desviaciones ocurridas.

Modelación. Se emplea para la construcción de los modelos de aprendizaje automático que representan el comportamiento de las variables operativas y generan las predicciones, y para el diseño de la arquitectura del sistema.

Enfoque de sistemas. Se utiliza para analizar el ERP de ISI Mustang como un sistema con entradas, procesos y salidas, identificando las interrelaciones entre sus módulos, los flujos de datos relevantes para el módulo predictivo y los puntos de integración con los componentes nuevos.

En cuanto al desarrollo del sistema, se adopta una metodología iterativa e incremental, en la que el producto se construye en ciclos sucesivos que agregan funcionalidad de manera progresiva y permiten validar tempranamente cada entrega con los usuarios de la empresa. El trabajo se organiza en cinco incrementos: la reimplementación en el ERP de los procesos del portal anterior; la autenticación centralizada con Keycloak; el pipeline de datos y el entrenamiento de los modelos; el microservicio de predicción y su integración con el ERP mediante el dashboard; y la integración con Power BI y el servidor MCP. Cada incremento comprende la definición de su alcance, el análisis, el diseño, el desarrollo y la validación, y se documenta en el Capítulo IV. Los fundamentos de esta metodología se desarrollan en el Capítulo II.

1.5.3 TÉCNICAS E INSTRUMENTOS DE INVESTIGACIÓN

Se emplean tres técnicas, cada una con sus instrumentos.

Análisis de datos. Se aplica sobre los registros históricos del portal anterior y las planillas de control complementarias, con el propósito de caracterizar las fuentes, identificar variables predictoras, detectar patrones de comportamiento y construir los conjuntos de datos necesarios para el entrenamiento de los modelos. Sus instrumentos son los scripts de extracción y transformación de datos del pipeline y los cuadernos de análisis exploratorio (*notebooks*).

Entrevista. Se aplica a usuarios clave del ERP en ISI Mustang Bolivia en la etapa de evaluación y validación del sistema, con el fin de validar las predicciones generadas y recoger criterios de utilidad desde la perspectiva operativa; sus resultados se presentan en el Capítulo IV. Los

requisitos funcionales del módulo, en cambio, no dependen de estas entrevistas: provienen del relevamiento de negocio ya documentado en la especificación del ERP, en particular de las preguntas del requerimiento registradas para cada proceso, y de la entrevista preliminar realizada en la etapa de perfil, y se presentan en el Capítulo III. El instrumento de las entrevistas de validación es una guía semiestructurada organizada en bloques: contexto del rol y gestión de proyectos; desviaciones y riesgos frecuentes; uso del sistema y datos disponibles; calidad y confiabilidad del registro; y utilidad y expectativas respecto al módulo predictivo.

Pruebas de los modelos predictivos. Se aplican sobre un conjunto de datos de prueba independiente del de entrenamiento para evaluar el desempeño de los modelos. Para los casos de clasificación se emplean la exactitud, la precisión, la sensibilidad, el F1 y la matriz de confusión; para los casos de regresión, la raíz del error cuadrático medio (RMSE), el error absoluto medio (MAE) y el coeficiente de determinación (R^2). Adicionalmente se aplica validación cruzada para verificar la estabilidad de los resultados. Sus instrumentos son el cuadro de métricas de evaluación y las rutinas de validación cruzada.

1.5.4 POBLACIÓN Y MUESTRA

Análisis de datos. La población corresponde a la totalidad de los registros históricos generados en el Portal ISIMustang Bolivia durante su período operativo y a las planillas de control de proyectos complementarias. Se trabaja con el conjunto completo de datos disponibles en los módulos relevantes para los casos de predicción definidos, sin muestreo, dado que el volumen es tratable en su totalidad por el pipeline analítico. Los proyectos activos registrados en el ERP nuevo constituyen la población sobre la que se realiza la inferencia.

Entrevista. La población está conformada por aproximadamente 20 usuarios del ERP en ISI Mustang Bolivia. La muestra es de 5 a 8 usuarios clave seleccionados mediante muestreo no probabilístico intencional, priorizando los perfiles con roles de jefe de proyecto, certificador y administrador del sistema, por ser quienes registran, controlan y consumen la información sobre la que opera el módulo predictivo.

1.6 DELIMITACIÓN DE LA INVESTIGACIÓN

1.6.1 DELIMITACIÓN TEMÁTICA

El presente trabajo se delimita a las áreas de aprendizaje automático supervisado, ingeniería de datos y analítica predictiva, aplicadas sobre los datos históricos operativos de ISI Mustang, y a la ingeniería de software aplicada a la modernización de su ERP: autenticación centralizada con Keycloak (OAuth 2.0 / OpenID Connect), integración con Power BI, exposición de datos mediante un servidor MCP y reimplementación en el ERP de los procesos del portal anterior. Los datos de entrenamiento provienen del sistema legado (Portal ISIMustang Bolivia, MySQL) y de las planillas de control de proyectos que se llevan fuera del sistema; la inferencia se realiza sobre los proyectos activos del ERP nuevo (PostgreSQL). El pipeline de datos es exclusivamente analítico: no migra datos al ERP. Los reportes en Power BI son descriptivos y no incorporan predicciones, que se presentan únicamente en el dashboard del ERP. En cuanto a la gestión de proyectos, el estudio se circunscribe a los proyectos de tipo EPC ejecutados por ISI Mustang.

Quedan fuera del alcance las fuentes de datos externas, como sensores IoT, datos de mercado o información climática; los módulos del sistema que no se encuentren activos o poblados con datos históricos suficientes; la migración del histórico del sistema legado al esquema del ERP nuevo; y el desarrollo de un asistente conversacional o agente de inteligencia artificial sobre el servidor MCP, que se plantea como trabajo futuro.

1.6.2 DELIMITACIÓN ESPACIAL

El estudio se desarrolla en las instalaciones de ISI Mustang Bolivia, con sede en Santa Cruz de la Sierra, Bolivia. Los datos históricos utilizados corresponden a proyectos ejecutados por la empresa principalmente en territorio boliviano. El módulo de inteligencia predictiva se integra y valida en el ERP operativo de ISI Mustang en su sede central, desplegado en un servidor propio de la empresa; la migración a la nube se plantea como trabajo futuro.

1.6.3 DELIMITACIÓN TEMPORAL

Los datos históricos utilizados para el entrenamiento de los modelos abarcan un período de más de diez años de operación de ISI Mustang, registrados en el sistema legado y en las planillas de control complementarias. El desarrollo del proyecto de grado se inicia en abril de 2026 y comprende las etapas de análisis, diseño, construcción por incrementos y evaluación del sistema, con un cierre sujeto al calendario académico de la carrera.

En cuanto a la validez temporal de la investigación, dado que los modelos predictivos se entrenan sobre patrones históricos, se estima que la solución mantiene su vigencia mientras las condiciones operativas de la empresa permanezcan estables. Se recomienda un reentrenamiento periódico de los modelos, de forma anual o ante cambios sustanciales en los procesos de la empresa, con el fin de mitigar la pérdida de precisión por deriva de datos (*model drift*).

CAPÍTULO II

MARCO TEÓRICO Y TECNOLÓGICO

2.1 SISTEMAS ERP

2.1.1 DEFINICIÓN Y CARACTERÍSTICAS

Los sistemas de planificación de recursos empresariales (*enterprise resource planning*, ERP) no cuentan con una definición única. **Klaus2000<empty citation>** encontraron visiones divergentes sobre el fenómeno y concluyeron que era poco probable alcanzar una definición ampliamente consensuada. Existe, sin embargo, un núcleo común. **Davenport1998<empty citation>** los describe como paquetes de software comerciales que prometen la integración fluida de toda la información que circula por una empresa: la financiera y contable, la de recursos humanos, la de la cadena de suministro y la de los clientes. **Shi2003<empty citation>** lo formulan en términos operativos: un entorno integrado que permite articular las principales funciones de gestión sobre una única base de datos común, de modo que la información se comparta entre ellas.

Ese núcleo se explica por su origen. El ERP nació en la industria manufacturera (**Shi2003**); **Jacobs2007<empty citation>** reconstruyen esa historia y el papel de los principales proveedores. **Klaus2000<empty citation>** sitúan el origen del término en la planificación de requerimientos de materiales (MRP) y la planificación de recursos de manufactura (MRP II): cuando las empresas advirtieron que la rentabilidad y la satisfacción del cliente eran objetivos de toda la organización, la integración se extendió a finanzas, ventas, distribución y recursos humanos, y a esa solución integral se la llamó ERP.

De esa caracterización se desprenden tres rasgos. El primero es la integración: una sola base de datos elimina la duplicación y la inconsistencia entre sistemas heredados dispersos, el atractivo que **Davenport1998<empty citation>** identifica frente a sistemas incompatibles. El segundo es la organización en módulos funcionales que comparten esa base y pueden adoptarse de forma progresiva. El tercero es la estandarización de procesos: el paquete trae incorporada una forma de operar y, como advierte el mismo autor, impone su propia lógica sobre la estrategia, la organización y la cultura de la empresa, por lo que su adopción debe ser responsabilidad de la gerencia general y no solo de los tecnólogos.

Los beneficios y los riesgos están documentados. **Shang2002<empty citation>**, a partir de 233 casos y entrevistas con gerentes de 34 organizaciones, consolidan los beneficios en cinco dimensiones (operacional, gerencial, estratégica, de infraestructura de TI y organizacional) y los evalúan en perspectiva longitudinal, años después de la implementación. Del lado de los riesgos, **Umble2003<empty citation>** describen al ERP como un sistema de información altamente complejo, cuya implementación es difícil y costosa, y señalan que muchas implementaciones han sido calificadas como fracasos por no alcanzar los objetivos corporativos predeterminados.

Frente al paquete comercial existe la alternativa del desarrollo propio. **Olsen2007<empty citation>** sostienen que para muchas pequeñas y medianas empresas el software desarrollado en casa es una opción óptima: ofrece control total, flexibilidad y adaptación a requisitos específicos, con costos a menudo comparables a los de un ERP estándar. Cuando el software soporta el núcleo del negocio, el desarrollo propio otorga ventaja competitiva, mientras que las estructuras rígidas de los paquetes pueden amenazar la dinámica organizacional. Es la situación de ISI Mustang: sus procesos centrales (centros de costo, certificaciones por avance, requisiciones, viáticos) se apoyan en un ERP desarrollado a medida, que hoy se reconstruye sobre NestJS y PostgreSQL a partir de un portal anterior en PHP y MySQL, y que este trabajo toma a la vez como objeto de mejora y como fuente de datos.

2.1.2 ERP EN EMPRESAS DE INGENIERÍA

La aplicación del ERP a las empresas de ingeniería y construcción exige revisar el supuesto manufacturero. **Shi2003<empty citation>** advierten que la naturaleza particular de la industria de la construcción impide adoptar directamente los sistemas orientados a manufactura y proponen las características fundamentales de un ERP para construcción: funciones de gestión de proyectos, técnicas de planificación avanzada y estandarización de procesos. En estas empresas la unidad de gestión deja de ser el producto repetitivo y pasa a ser el proyecto, con alcance, presupuesto, plazo y cliente propios, y el sistema debe seguir cada uno por separado sin perder la visión consolidada de la empresa.

Aun con esas adaptaciones, la adopción en el sector no es sencilla.

Chung2008<empty citation> reconocen que los ERP ofrecen muchos beneficios a la industria de ingeniería y construcción, pero que muchas firmas dudan en adoptarlos por su alto costo, sus incertidumbres y sus riesgos; en respuesta identifican factores críticos de éxito y fracaso e indicadores para evaluar el éxito del ERP, y proponen un modelo que los relaciona. El trabajo traslada al sector la discusión genérica de **Umble2003<empty citation>**, y su enfoque asume que el resultado de una implementación depende de factores organizacionales y no solo técnicos.

Una segunda línea conecta el ERP con el control de desempeño de los proyectos. **Skibniewski2009<empty citation>** se preguntan qué áreas de proceso de la construcción no pueden ser cubiertas por el ERP a la hora de recolectar indicadores clave de desempeño (KPI), y lo responden con una encuesta a los responsables de ERP de grandes firmas de ingeniería y construcción de Estados Unidos. El planteamiento del trabajo supone que una parte de los procesos de proyecto queda fuera del alcance de un ERP estándar, lo que refuerza el argumento de **Olsen2007<empty citation>** a favor del desarrollo propio cuando el núcleo del negocio es el proyecto.

Lo anterior se corresponde con la forma en que ISI Mustang ha organizado su propio sistema. En el ERP de la empresa la unidad que articula toda la operación es el centro de costo, que puede representar un proyecto, un servicio o un área de estructura y tiene código, responsable, moneda y vigencia propios. Sobre esa unidad se registran las horas del personal, las requisiciones y las órdenes de compra emitidas a los proveedores, los viáticos y sus rendiciones, y el presupuesto, con una línea base separada del seguimiento. El ingreso ejecutado de cada proyecto se registra mediante certificaciones, cuyo flujo distingue la solicitud por parte del gerente del centro de costo, el registro y el cobro.

El portal anterior, cuyos procesos se analizan en el Capítulo III, calculaba además indicadores de seguimiento y control por proyecto, entre ellos el desvío de costo, el desvío de ingresos, el avance del proyecto y el avance de trabajo, a partir del monto de la orden de compra emitida por el cliente, lo ejecutado y el flujo de caja planificado. Esos indicadores son el antecedente directo de las desviaciones que el módulo predictivo busca anticipar, y los datos con los que se calculaban son los que se exploran en el Capítulo III.

2.1.3 ERP EN EL SECTOR DE PETRÓLEO, GAS Y MINERÍA

La literatura sobre ERP en petróleo, gas y minería es más escasa que la de manufactura o construcción y está compuesta sobre todo por estudios de caso, cercanos a los sectores en los que ISI Mustang ejecuta sus proyectos. En petróleo y gas, Al Jafari y Nair (2018) analizan, con un enfoque mixto, cómo la implementación del ERP influye en la eficiencia y la productividad de las empresas del sector en el Sultanato de Omán. Parten de que el papel del ERP en esa industria no puede negarse y concluyen recomendando su implementación como palanca de crecimiento.

Los retos organizacionales aparecen con más nitidez en **Ali2023<empty citation>**, que mediante investigación-acción estudian las barreras culturales a la implementación de ERP en el sector de petróleo y gas de Medio Oriente, desde la planificación hasta la posimplementación. Identifican como causas del retraso el conflicto entre equipos, la autoridad gerencial, la falta de una cultura de TI y de compromiso con la capacitación, y la tecnofobia. **Gool2018<empty citation>**, por su parte, plantean el dilema de cuándo personalizar el ERP y cuándo cambiar los procesos de negocio para ajustarlos a la funcionalidad estándar. En una petrolera multinacional africana encuentran que la personalización continúa después de la implementación y describen prácticas para gestionarla: capacitación, eliminación sistemática de modificaciones y procesos de aprobación.

Para la minería, **Larasati2023<empty citation>** encuestan a la gerencia y el personal de una empresa minera indonesia y ordenan los factores críticos de fracaso del ERP con el método TOPSIS (*Technique for Order of Preference by Similarity to Ideal Solution*); los dos más importantes resultan ser la mala comprensión de los procesos de negocio de la organización y una reingeniería de procesos deficiente. El hallazgo coincide con lo observado en petróleo y gas: en las industrias extractivas el ERP fracasa o se retrasa menos por la tecnología que por el conocimiento de los procesos, la cultura de uso y la gestión continua de la adaptación. Cabe suponer que lo mismo vale para las empresas EPC que trabajan para ellas, aunque ninguno de los casos revisados las estudia directamente.

Tres lecciones de esa literatura orientan este proyecto: documentar los flujos de negocio antes de sistematizarlos, criterio que se sigue en el análisis de procesos del portal anterior;

asumir la personalización como un proceso continuo, lo que es coherente con la metodología iterativa e incremental de la construcción; y anticipar que la resistencia al uso señalada por **Ali2023**<empty citation> puede afectar también a un módulo predictivo, de ahí la validación con usuarios clave prevista en el Capítulo IV.

Queda, por último, el papel del ERP que más interesa a este trabajo: el de fuente de datos. Un sistema que durante más de diez años ha registrado horas, compras, presupuestos e ingresos facturados por proyecto, complementado con las planillas de control que la empresa lleva fuera del sistema, acumula un histórico del que pueden aprenderse patrones. Jawad y Villányi (2024) revisan la integración del aprendizaje automático con los sistemas ERP: sus algoritmos extraen patrones complejos de grandes conjuntos de datos y permiten pronósticos más precisos, con un énfasis creciente en la interpretabilidad de los resultados. Ese es el enlace con las secciones que siguen: el aprendizaje automático aporta los métodos, la analítica predictiva el marco de aplicación, y el ERP de ISI Mustang, tanto el histórico del portal anterior en MySQL como los proyectos activos del ERP nuevo en PostgreSQL, los datos.

2.2 APRENDIZAJE AUTOMÁTICO

2.2.1 TIPOS DE APRENDIZAJE

El aprendizaje automático (*machine learning*) es la rama de la inteligencia artificial que estudia los algoritmos capaces de mejorar su desempeño de manera automática a partir de la experiencia. **Mitchell1997**<empty citation> lo formaliza con tres elementos: se dice que un programa aprende de una experiencia E respecto de una clase de tareas T y una medida de desempeño P si su desempeño en T , medido por P , mejora con E . Trasladada al presente proyecto, esa formalización ubica la experiencia en el histórico operativo de más de diez años de ISI Mustang y la tarea en anticipar desviaciones en sus proyectos, mientras que la medida de desempeño está dada por las métricas que se describen en la subsección siguiente.

La forma en que esa experiencia se presenta al algoritmo define los paradigmas clásicos

del campo. **Geron2022**^{<empty citation>} organiza los sistemas de aprendizaje automático según la supervisión que reciben durante el entrenamiento en tres familias: supervisado, no supervisado y por refuerzo. En el aprendizaje supervisado cada observación de entrenamiento viene acompañada de la respuesta correcta, la etiqueta o valor objetivo, y el algoritmo aprende una función que asocia las entradas con esa salida para predecirla sobre observaciones nuevas (**Hastie2009**). En el aprendizaje no supervisado no existe tal respuesta: el algoritmo recibe únicamente las entradas y busca estructura en ellas, como grupos de observaciones similares o direcciones de mayor variabilidad (**James2021**). El aprendizaje por refuerzo, por último, no parte de un conjunto de datos fijo sino de un agente que interactúa con un entorno y ajusta su comportamiento según las recompensas que recibe (**Mitchell1997**).

Dentro del aprendizaje supervisado, la naturaleza de la variable de salida distingue dos tipos de problema. Cuando la salida es cuantitativa (un monto, una duración, un porcentaje) se habla de regresión; cuando es cualitativa o categórica (una clase entre un conjunto finito) se habla de clasificación (**Hastie2009**; **James2021**). La distinción no es solo nominal: determina qué algoritmos son aplicables, qué función de pérdida se optimiza y con qué métricas se evalúa el resultado.

Los tres casos de uso del módulo predictivo se plantean en el paradigma supervisado: la predicción de desviación en certificaciones como un problema de clasificación, y la estimación del sobrecosto y del retraso en la fecha de cierre como problemas de regresión. El planteo supone que el histórico de la empresa contiene, para cada proyecto cerrado, el resultado que se quiere predecir: cuánto costó respecto de lo presupuestado, cuándo se cerró respecto de la fecha comprometida y cómo evolucionaron sus certificaciones frente a lo planificado. La existencia, completitud y recuperabilidad de esas respuestas para una proporción suficiente de proyectos se examina en el análisis de datos del Capítulo III, del que dependen también los detalles de las variables y del protocolo de evaluación que esta sección presenta en términos generales. El aprendizaje no supervisado y el aprendizaje por refuerzo quedan fuera del alcance del proyecto.

2.2.2 APRENDIZAJE SUPERVISADO

Formalmente, el aprendizaje supervisado parte de un conjunto de entrenamiento compuesto

por pares (x_i, y_i) , donde x_i es un vector de variables predictoras y y_i la respuesta observada, y busca una función f tal que $f(x)$ aproxime a y con el menor error posible sobre observaciones que no participaron del entrenamiento (**Hastie2009**). Esta última condición es la que separa aprender de memorizar. **James2021**<empty citation> distinguen el error de entrenamiento, que mide el ajuste sobre los datos vistos, del error de prueba, que mide la capacidad de generalizar; un modelo puede reducir el primero indefinidamente aumentando su complejidad, pero a partir de cierto punto el segundo empieza a crecer. Ese fenómeno es el sobreajuste, y se explica por el compromiso entre sesgo y varianza: los modelos simples tienen sesgo alto porque no capturan la estructura de los datos, mientras que los modelos muy flexibles tienen varianza alta porque se ajustan al ruido de la muestra particular con la que fueron entrenados (**Hastie2009**; **James2021**).

Como el error de prueba no puede calcularse sobre los datos de entrenamiento, se requiere un procedimiento para estimarlo. La práctica básica consiste en separar una parte de los datos como conjunto de prueba, que se reserva y solo se usa al final para evaluar el modelo elegido (**Geron2022**). Para seleccionar entre modelos o ajustar hiperparámetros sin contaminar ese conjunto se emplea la validación cruzada de k particiones (*k-fold*): los datos de entrenamiento se dividen en k bloques, el modelo se entrena k veces dejando fuera un bloque distinto en cada corrida y el error se promedia sobre las k evaluaciones (**James2021**). En un estudio empírico de referencia sobre selección de clasificadores, **Kohavi1995**<empty citation> concluye que la validación cruzada estratificada de diez particiones es el mejor método para selección de modelos, incluso cuando la capacidad de cómputo permite usar más particiones, y advierte que la variante *leave-one-out*, aunque casi insesgada, tiene alta varianza y produce estimaciones poco confiables. La estratificación conserva en cada bloque la proporción de clases del conjunto completo; es propia de los problemas de clasificación e importa cuando una de las clases es minoritaria.

La evaluación del modelo depende del tipo de problema. En clasificación, las medidas usuales derivan de la matriz de confusión, que cruza la clase real con la clase predicha (**Sokolova2009**; **Geron2022**); de ella se obtienen la exactitud, la precisión, la sensibilidad o *recall* y la medida F1, definidas en el cuadro 2.1. **Sokolova2009**<empty citation> analizan qué cambios en la matriz de confusión dejan invariante cada medida y muestran que estas cuatro reaccionan de forma distinta ante cambios en la distribución de clases, por lo que conviene reportarlas juntas en lugar de limitarse a la exactitud.

En regresión, el punto de partida es el error cuadrático medio (MSE), que al elevar al cuadrado las diferencias entre valor observado y predicho da más peso a los errores grandes (**James2021**); su raíz, el RMSE, se expresa en las unidades de la variable, mientras que el error absoluto medio (MAE), al promediar diferencias en valor absoluto, penaliza menos esos errores. **Chai2014<empty citation>** sostienen que el RMSE es más apropiado cuando se espera que los errores sigan una distribución gaussiana y que evaluar un modelo requiere una combinación de métricas y no una sola. El coeficiente de determinación R^2 mide la proporción de la variabilidad de la respuesta que el modelo explica; toma la forma de una proporción y no depende de la escala de la variable (**James2021**), aunque qué valor se considera bueno depende de la aplicación. El cuadro 2.1 resume las métricas y su uso en el proyecto.

Cuadro 2.1. Métricas de evaluación de modelos supervisados y su uso en el proyecto

Métrica	Qué mide	Uso en el proyecto
<i>Clasificación</i>		
Matriz de confusión	Cruce de clase real y clase predicha (VP, VN, FP, FN)	Base de las demás métricas; muestra qué tipo de error comete el clasificador de desviación en certificaciones
Exactitud	$(VP + VN)/total$	Referencia general; insuficiente si las clases están desbalanceadas
Precisión	$VP/(VP + FP)$	Proporción de alertas de desviación que son correctas
Sensibilidad	$VP/(VP + FN)$	Proporción de desviaciones reales que el modelo detecta
F1	Media armónica de precisión y sensibilidad	Resumen único cuando ambas importan
<i>Regresión</i>		
RMSE	Raíz del promedio de errores al cuadrado	Error en unidades de la variable; penaliza sobrecostos o retrasos grandes mal estimados
MAE	Promedio de errores en valor absoluto	Error típico esperado, menos sensible a valores extremos
R^2	Proporción de la variabilidad explicada por el modelo	Ajuste de cada regresor sobre su propia respuesta, independiente de la escala; complementa al RMSE y al MAE

Fuente: Elaboración propia a partir de Sokolova2009<empty citation>, James2021<empty citation> y Chai2014<empty citation>.

Con estas definiciones, el protocolo de evaluación del módulo predictivo es el siguiente: el clasificador de desviación en certificaciones se evalúa con exactitud, precisión, sensibilidad, F1 y matriz de confusión; los regresores de sobrecosto y de retraso en la fecha de cierre, con RMSE, MAE y R^2 . En los tres casos la selección de hiperparámetros se realiza con validación cruzada de

k particiones sobre el conjunto de entrenamiento (estratificada solo en el clasificador) y se reserva un conjunto de prueba independiente para la evaluación final.

2.2.3 ALGORITMOS DE APRENDIZAJE SUPERVISADO: GRADIENT BOOSTING Y XGBOOST

El modelo base de los algoritmos empleados en este proyecto es el árbol de decisión. Un árbol particiona recursivamente el espacio de las variables predictoras mediante reglas binarias del tipo «presupuesto mayor que cierto umbral» y asigna a cada región resultante una predicción constante: la clase mayoritaria en clasificación o el promedio de las respuestas en regresión (Hastie2009; James2021). El procedimiento CART de Breiman1984 es la formulación de referencia de este tipo de árboles binarios. Los árboles son fáciles de interpretar y, en su formulación teórica, pueden dividir directamente sobre variables categóricas; las implementaciones actuales de *gradient boosting* admiten esa capacidad de forma nativa como opción, XGBoost desde su versión 1.5 y Scikit-learn en sus estimadores basados en histogramas. En la configuración adoptada en este proyecto, sin embargo, los predictores categóricos se codifican de forma explícita antes del entrenamiento, como se describe en la subsección siguiente, lo que deja la representación de las variables bajo el control del pipeline de datos y no de la biblioteca. Por otra parte, un árbol aislado tiene alta varianza (pequeños cambios en los datos producen árboles muy distintos) y su capacidad predictiva suele ser inferior a la de otros métodos (James2021).

Los métodos de ensamble corrigen esa debilidad combinando muchos árboles, con dos estrategias distintas. En el *bagging*, los árboles se entrenan en paralelo sobre remuestreos de los datos y sus predicciones se promedian, lo que reduce la varianza; los bosques aleatorios de Breiman2001 parten de esa idea y añaden una selección aleatoria de variables en cada división. En el *boosting*, en cambio, los árboles se construyen de forma secuencial: cada nuevo árbol se ajusta a lo que los anteriores todavía no explican, de modo que el ensamble mejora por etapas (Hastie2009; James2021).

Friedman2001 da a esta idea su formulación general al plantear la estimación de la función como un problema de optimización numérica en el espacio de funciones:

en cada etapa se ajusta un árbol de regresión al gradiente negativo de la función de pérdida evaluada sobre el modelo acumulado, lo que equivale a un descenso por gradiente en el que cada paso es un árbol. El marco admite distintas funciones de pérdida (mínimos cuadrados, desviación absoluta o Huber para regresión, y verosimilitud logística para clasificación), de modo que el mismo algoritmo sirve para los problemas de regresión y de clasificación del proyecto. Sus hiperparámetros principales son el número de árboles, la tasa de aprendizaje o *shrinkage*, que reduce la contribución de cada árbol, y la profundidad de los árboles, que controla el orden de las interacciones entre variables (**James2021**); el submuestreo de observaciones en cada etapa agrega una regularización adicional (**Hastie2009**).

XGBoost es la implementación de *gradient boosting* de árboles que se prevé utilizar. **Chen2016** la presentan como un sistema escalable, ampliamente usado para obtener resultados de vanguardia en competencias de aprendizaje automático, y describen sus aportes: un algoritmo *sparsity-aware* para datos dispersos, un esquema de cuantiles ponderados para construir árboles de forma aproximada, y optimizaciones de caché, compresión y particionado que le permiten escalar a miles de millones de ejemplos con menos recursos que otros sistemas.

Dos características de XGBoost interesan al diseño del módulo predictivo (**Chen2016**). La primera es el tratamiento de valores ausentes: el algoritmo de búsqueda de divisiones *sparsity-aware* incorpora en cada nodo una dirección por defecto, aprendida de los datos, hacia la que se envían las observaciones que carecen del valor de la variable de división, lo que permite entrenar sin imputar previamente los valores ausentes. La segunda es la función objetivo regularizada, que suma a la pérdida un término de complejidad que penaliza el número de hojas y la magnitud de los pesos de cada árbol; sus coeficientes son hiperparámetros adicionales a los del *boosting* ya mencionados y se ajustan mediante validación cruzada.

La elección de *gradient boosting* frente a alternativas más recientes se apoya en evidencia sobre datos tabulares. **Grinsztajn2022** comparan modelos de árboles y redes neuronales profundas sobre 45 conjuntos de datos tabulares y concluyen que los primeros siguen siendo el estado del arte en conjuntos de tamaño medio, del orden de diez mil observaciones, aun sin considerar su ventaja en velocidad; atribuyen el resultado a sesgos inductivos de los árboles, como la robustez frente a variables no informativas y la capacidad de aprender funciones

irregulares, que las redes no comparten. **ShwartzZiv2022<empty citation>** llegan a la misma conclusión al enfrentar XGBoost con modelos profundos diseñados específicamente para datos tabulares: XGBoost los supera incluso en los conjuntos de datos usados por los artículos que los proponen y requiere mucho menos ajuste. Ambos resultados se obtienen sobre datos tabulares, la forma que toma el histórico de la empresa, y respaldan la elección de XGBoost para los tres casos de uso. En la implementación se emplea su interfaz compatible con Scikit-learn, la biblioteca de aprendizaje automático en Python presentada por **Pedregosa2011<empty citation>**. Cabe una salvedad: los conjuntos evaluados en esos estudios son del orden de miles de observaciones, mientras que el histórico de la empresa, con un vector de variables por proyecto, probablemente reúne bastantes menos, lo que condiciona hasta qué punto esa evidencia es trasladable.

2.2.4 FEATURE ENGINEERING

La ingeniería de características (*feature engineering*) es el conjunto de operaciones que transforman los datos crudos en las variables predictoras que recibe el modelo. **Kuhn2019<empty citation>** la sitúan dentro del proceso de modelado predictivo, junto con la división de datos, el remuestreo, el ajuste de hiperparámetros y la evaluación, y sostienen que la representación de los predictores condiciona el desempeño tanto como la elección del algoritmo. **Geron2022<empty citation>** muestra el mismo proceso desde la práctica con Scikit-learn: limpieza, codificación de variables categóricas, escalado y encadenamiento de esas transformaciones en *pipelines* que se ajustan únicamente con datos de entrenamiento, para evitar que información del conjunto de prueba se filtre al modelo.

Tres grupos de técnicas son relevantes para este proyecto. El primero es la codificación de predictores categóricos, como el cliente, el tipo de proyecto o el rubro, que en este proyecto se realiza de forma explícita en el pipeline según la decisión indicada en la subsección anterior: **Kuhn2019<empty citation>** describen la codificación *one-hot* o por variables indicadoras, el agrupamiento de categorías poco frecuentes y las codificaciones supervisadas, que reemplazan cada categoría por un estadístico de la respuesta, útiles cuando la variable tiene muchos niveles. El segundo es la transformación de predictores numéricos, que incluye el centrado y escalado, las transformaciones para corregir asimetría y la discretización en intervalos. El tercero, y el

más sensible en un histórico operativo, es el manejo de datos faltantes: los mismos autores recomiendan visualizar primero el patrón de ausencia y luego decidir entre eliminar observaciones o variables, imputar los valores o emplear modelos tolerantes a faltantes. Esta última opción se corresponde con el tratamiento de valores ausentes de XGBoost descrito en la subsección anterior.

La selección de variables completa el proceso. **Guyon2003<empty citation>** le atribuyen tres objetivos: mejorar el desempeño del predictor, obtener modelos más rápidos y económicos, y comprender mejor el proceso que generó los datos. **Kuhn2019<empty citation>** clasifican los métodos en filtros, que evalúan cada variable de forma independiente del modelo, y envoltentes (*wrappers*), que evalúan el modelo sobre subconjuntos candidatos de variables, ya sea mediante búsquedas voraces, como la eliminación recursiva, o mediante búsquedas globales; y advierten que, si la selección se realiza fuera del esquema de remuestreo, la estimación del error queda sesgada de forma optimista.

En el proyecto estas operaciones constituyen una etapa propia del pipeline de datos, que construye para cada proyecto un vector de variables a partir de sus atributos maestros y de agregados de sus registros transaccionales a una fecha de corte. Un requisito específico es que las mismas variables puedan calcularse tanto desde el histórico del portal anterior, que alimenta el entrenamiento, como desde el ERP nuevo, sobre el que se realiza la inferencia; el diseño de esa etapa se desarrolla en el Capítulo III.

2.3 INGENIERÍA DE DATOS

2.3.1 PROCESOS ETL Y ELT

Los procesos de extracción, transformación y carga (ETL, por *extract, transform, load*) son el mecanismo clásico para alimentar un almacén de datos desde los sistemas operacionales. **Vassiliadis2009<empty citation>** los define como los procesos de software que facilitan la carga original y el refresco periódico del contenido del almacén, y revisa los problemas propios de cada una de las tres etapas y los que afectan al proceso completo. El destino es el almacén de datos

que **Inmon2005** caracteriza como una colección de datos orientada a temas, integrada, variable en el tiempo y no volátil, en apoyo de la toma de decisiones; la integración y la orientación a temas son, precisamente, el trabajo que ETL realiza sobre datos que en origen están organizados por aplicación y por transacción.

Desde la práctica del modelado dimensional, **Kimball2013** advierten que el sistema ETL consume una parte desproporcionada del tiempo y del esfuerzo necesarios para construir un entorno de almacén de datos e inteligencia de negocios, y lo descomponen en 34 subsistemas agrupados en cuatro áreas: extraer los datos de las fuentes, limpiarlos y conformarlos, entregarlos preparados para la presentación y gestionar el entorno ETL. Dos de sus aportes interesan a este proyecto. El primero es el conformado (*conforming*): lograr que atributos equivalentes de fuentes distintas queden representados con los mismos nombres, dominios y significados. El segundo es la distinción entre la carga histórica única, que puebla el almacén con todo el pasado disponible, y el procesamiento incremental, que lo actualiza periódicamente con los registros nuevos.

La variante ELT (*extract, load, transform*) invierte las dos últimas etapas: los datos se cargan crudos en el destino y la transformación se ejecuta allí, con el motor de consulta del propio almacén o lago. Reis y Housley (2022) ubican ambos patrones en la etapa de transformación de un ciclo de vida de la ingeniería de datos que va de la generación a la ingesta, el almacenamiento, la transformación y el servicio; explican que ETL surgió cuando el cómputo era un recurso escaso y convenía transformar antes de cargar, que ELT es el patrón propio de los lagos de datos y *lakehouses* modernos, y recomiendan elegir según el caso en lugar de adherir a uno por principio. **Armbrust2021** muestran el costo de no hacerlo: en la arquitectura de dos niveles habitual en las empresas los datos primero se someten a ETL hacia el lago y luego a ELT hacia el almacén, y cada paso adicional agrega demoras, complejidad y nuevas oportunidades de introducir errores que reducen la calidad de los datos.

El pipeline de datos de este proyecto es un caso de ELT con fines exclusivamente analíticos. Las fuentes son la base MySQL del portal anterior, la base PostgreSQL del ERP nuevo y las planillas Excel de control de proyectos que la empresa lleva fuera del sistema; sus datos se copian sin modificaciones a una capa cruda y se transforman después, en las capas que describe la

subsección siguiente. La distinción de Kimball y Ross se traduce en dos modos de operación: una carga histórica del portal anterior y de las planillas, que produce el conjunto de entrenamiento, y una carga incremental desde el ERP nuevo, que alimenta la inferencia sobre los proyectos activos. En ningún caso el pipeline escribe de vuelta en el ERP ni migra el histórico al esquema nuevo; esa migración queda fuera del alcance del trabajo.

2.3.2 ARQUITECTURA MEDALLION (BRONZE, SILVER, GOLD)

El principio de organizar los datos en capas de calidad creciente tiene su fundamento académico en la propuesta de *lakehouse* de **Armbrust2021**^{<empty citation>}. Los autores parten de un diagnóstico: los lagos de datos de segunda generación, que aplican el esquema en el momento de la lectura, trasladaron aguas abajo el problema de la calidad y la gobernanza, y combinarlos con un almacén separado produce datos desactualizados, poca fiabilidad y un alto costo total. En respuesta proponen una plataforma única que almacena los datos en formatos abiertos de acceso directo, como Parquet, ofrece soporte de primera clase al aprendizaje automático y a la ciencia de datos, y alcanza un rendimiento comparable al de un almacén. Dos rasgos de esa propuesta importan aquí: un *lakehouse* debe poder guardar los datos crudos, como un lago, y al mismo tiempo soportar procesos ETL/ELT que los curen para mejorar su calidad con vistas al análisis; y su capa de metadatos es el lugar natural para aplicar controles de calidad, como la imposición de esquema y las restricciones que rechazan o ponen en cuarentena los registros inválidos.

La nomenclatura con la que hoy se designa ese curado por etapas no proviene de ese artículo, sino de la documentación de Databricks, es decir, de literatura de proveedor. Databricks (2025) define la arquitectura *medallion*, también llamada de múltiples saltos (*multi-hop*), como un patrón de diseño para organizar lógicamente los datos en una serie de capas que denotan la calidad de lo almacenado. La capa Bronze recibe los datos crudos de las fuentes sin validación y conserva su estado original; la capa Silver contiene al menos una representación validada y no agregada de cada registro, y es donde se realizan la limpieza, la deduplicación y la normalización; la capa Gold consiste en datos agregados y ajustados a los requisitos de análisis y reporte. Al hacer progresar los datos por las capas, su calidad y fiabilidad mejoran de manera incremental hasta hacerlos aptos

para inteligencia de negocios y aprendizaje automático. El patrón es conceptual y no prescribe un motor ni un formato.

El proyecto lo adopta sin la plataforma del proveedor, en línea con el argumento de **Armbrust2021<empty citation>** a favor de los formatos abiertos: se prevé materializar las capas como archivos en el servidor de la empresa y ejecutar las transformaciones con dos herramientas ligeras que se describen en la sección 2.7, DuckDB para Silver y Polars para Gold. El cuadro 2.2 resume la correspondencia. Bronze reúne copias crudas de las tres fuentes: las tablas de MySQL del portal anterior, las tablas de PostgreSQL del ERP nuevo y las planillas Excel. Silver es la capa en la que el conformado de **Kimball2013<empty citation>** se convierte en un requisito del modelo predictivo: los dos esquemas relacionales describen los mismos objetos de negocio (centros de costo, certificaciones, órdenes de compra, presupuestos) con nombres y estructuras distintos, y DuckDB los mapea a un conjunto único de variables con las mismas unidades y dominios, de modo que un modelo entrenado sobre el histórico pueda predecir sobre los proyectos activos. Gold construye con Polars las tablas de características por proyecto a una fecha de corte, único insumo de los modelos. El diseño detallado de cada capa se desarrolla en el Capítulo III.

Cuadro 2.2. Capas de la arquitectura *medallion* y su correspondencia en el proyecto

Capa	Contenido según el patrón	En el proyecto
Bronze	Datos crudos de las fuentes, sin validación, en su estado original	Copias de las tablas MySQL del portal anterior, de las tablas PostgreSQL del ERP nuevo y de las planillas Excel de control de proyectos
Silver	Al menos una representación validada y no agregada de cada registro; limpieza, deduplicación y normalización	Esquema unificado en DuckDB: los dos esquemas relacionales y las planillas mapeados a las mismas variables, con tipos y dominios conformados y registros deduplicados
Gold	Datos agregados y ajustados a los requisitos de análisis y reporte	Tablas de características por proyecto construidas con Polars; entrada del entrenamiento (histórico) y de la inferencia (proyectos activos)

Fuente: Elaboración propia a partir de Databricks (2025) y Armbrust2021<empty citation>.

2.3.3 CALIDAD DE DATOS

La calidad de los datos no se reduce a su exactitud. WangStrong1996<empty citation> observan que los esfuerzos de mejora tienden a concentrarse en la exactitud, mientras que los consumidores de datos manejan una conceptualización mucho más amplia, y construyen un marco jerárquico que organiza las dimensiones de calidad en cuatro categorías: intrínseca, las propiedades que los datos tienen por sí mismos, como la exactitud; contextual, que exige considerarlos en el contexto de la tarea en la que se usan e incluye la completitud y la oportunidad; representacional, referida a la forma en que se presentan, incluida la consistencia; y de accesibilidad. Datos de alta calidad son, en esa formulación, intrínsecamente buenos, apropiados para la tarea, claramente representados y accesibles. De la categoría contextual se sigue que la calidad de un conjunto de datos no se juzga en abstracto, sino respecto de la tarea a la que se destina.

Las dimensiones concretas más usadas son la exactitud, la completitud, la consistencia y la oportunidad, a las que **BatiniScannapieco2016<empty citation>** dedican un capítulo de su tratado junto con otras dimensiones temporales; el cuerpo de conocimiento de **DAMA2017<empty citation>** agrega, entre otras, la unicidad y la validez. No existe, sin embargo, un estándar único: los nombres y las definiciones varían entre autores, y **Batini2009<empty citation>** muestran que la literatura ofrece una amplia gama de metodologías de evaluación y mejora, que difieren en fases, estrategias y tipos de datos y de sistemas. El cuadro 2.3 fija las definiciones operativas que este trabajo adopta para seis dimensiones aplicables a un histórico relacional, y la verificación con que cada una se mide.

Cuadro 2.3. Dimensiones de calidad de datos adoptadas en el proyecto

Dimensión	Definición operativa	Verificación sobre el histórico
Exactitud	El valor registrado coincide con el valor real del objeto que describe	Contraste de montos y fechas del sistema contra las planillas de control y documentos de respaldo, sobre una muestra
Completitud	Los valores requeridos para la tarea están presentes	Proporción de proyectos con presupuesto, fechas y certificaciones registrados; patrón de valores nulos por variable
Consistencia	Los valores no se contradicen entre registros ni entre tablas	Sumas de detalle frente a totales de cabecera; estados de flujo compatibles con las fechas
Oportunidad	El dato está disponible cuando la tarea lo necesita	Rezago entre la fecha del evento y la fecha de registro en el sistema
Unicidad	Cada objeto de negocio aparece una sola vez	Duplicados de proyectos, clientes y proveedores por clave natural
Validez	Los valores pertenecen al dominio definido	Valores fuera de rango o de catálogo; tipos incorrectos en campos de texto libre

Fuente: Elaboración propia a partir de WangStrong1996<empty citation>, BatiniScannapieco2016<empty citation> y DAMA2017<empty citation>.

La relación entre esas dimensiones y el desempeño de un modelo de aprendizaje automático cuenta con evidencia empírica reciente. **Mohammed2025<empty citation>** sostienen que datos de entrenamiento incompletos, erróneos o inapropiados conducen a modelos poco confiables, y estudian de forma sistemática el efecto de seis dimensiones de calidad sobre el desempeño de 19 algoritmos de clasificación, regresión y agrupamiento sobre datos tabulares, contaminando los datos de entrenamiento, los de prueba o ambos; un diseño pertinente para un proyecto cuyos datos son tabulares y cuyos modelos se entrenan sobre un histórico y se aplican sobre datos nuevos. **Sambasivan2021<empty citation>** aportan la dimensión organizacional: a partir de entrevistas con 53 practicantes de inteligencia artificial en contextos de alto riesgo, describen las cascadas de datos, efectos negativos que se propagan aguas abajo a partir de problemas en los datos, y las encuentran omnipresentes, invisibles, de efecto retardado y con frecuencia evitables, fruto de prácticas que subvaloran los datos frente al modelado. **Gudivada2017<empty citation>** agregan que, para el aprendizaje automático, la calidad va más allá de la limpieza y las transformaciones y requiere gobernar el ciclo de vida completo de los datos.

De esa literatura el proyecto toma dos decisiones. La primera es que la exploración del histórico del Capítulo III usa las dimensiones del cuadro 2.3 como instrumento y las juzga respecto de la tarea predictiva concreta; qué grado de calidad tiene ese histórico en cada dimensión es un resultado de esa exploración y no se anticipa aquí. La segunda es que la mejora se aborda por dos vías: la limpieza en la capa Silver, que corrige los datos, y la corrección de los defectos del ERP que los generan, que corrige el proceso, de modo que los registros sobre los que se realiza la inferencia nazcan con mejor calidad que los históricos con los que se entrena.

2.4 ANALÍTICA PREDICTIVA

2.4.1 DEFINICIÓN Y APLICACIONES

La analítica de negocios es, en la formulación de **Davenport2007<empty citation>**, el uso extensivo de datos, de análisis estadístico y cuantitativo, de modelos explicativos y predictivos y de una gestión basada en hechos para orientar decisiones y acciones. Los mismos autores sostienen

que las empresas que compiten con éxito no se limitan a acumular datos, sino que construyen sus estrategias alrededor del conocimiento que extraen de ellos, combinando análisis cuantitativo, modelado predictivo, liderazgo orientado a los datos e infraestructura tecnológica. La definición interesa porque incluye de forma explícita a los modelos predictivos como un componente de la analítica, y no como una disciplina aparte.

Dentro de ese campo, la clasificación más difundida distingue tres niveles. **Delen2013** separan la analítica descriptiva, que resume lo que ocurrió, de la predictiva, que estima qué podría ocurrir a continuación, y de la prescriptiva, que busca los mejores pasos a seguir. **Lepenioti2020** confirman en su revisión que el grueso del trabajo académico e industrial se concentra todavía en los dos primeros niveles y presentan a la prescriptiva como el siguiente escalón de madurez analítica, orientado a anticipar decisiones optimizadas. La tríada sirve para ubicar con precisión lo que este proyecto construye: los reportes de Power BI cubren el nivel descriptivo sobre los datos del ERP, los modelos de aprendizaje automático cubren el predictivo, y el nivel prescriptivo, que supondría recomendar acciones correctivas, queda fuera del alcance como trabajo futuro.

Que el objetivo sea predecir tiene consecuencias metodológicas que no siempre se hacen explícitas. **Shmueli2010** observa que en muchas disciplinas domina el modelado explicativo, orientado a probar relaciones causales, y que con frecuencia se asume que un modelo con alto poder explicativo tiene también alto poder predictivo; la autora muestra que esa confusión es común y que la distinción afecta cada paso del proceso de modelado, desde el diseño y la recolección de datos hasta la selección de variables, la elección del método y la evaluación. Para este trabajo la implicancia es directa: los modelos se juzgan por su error sobre observaciones no vistas, con las métricas y la validación cruzada descritas en la sección 2.2, y no por la significancia o la interpretabilidad de sus coeficientes. Esa prioridad es la que justifica emplear *gradient boosting* aunque sea menos transparente que una regresión lineal.

La analítica predictiva aplicada a operaciones exige, además del método, conocimiento del negocio. **Waller2013**, al presentarla como una transformación del diseño y la gestión de cadenas de suministro, insisten en que la ciencia de datos requiere tanto dominio del área como un conjunto amplio de habilidades cuantitativas. En el contexto de los sistemas

empresariales, Jawad y Villányi (2024) describen la integración de algoritmos de aprendizaje automático en el ERP como el medio para producir predicciones más precisas y decisiones basadas en datos, con el requisito de que los modelos resulten comprensibles para los interesados. El presente proyecto sigue esa línea: el conocimiento del dominio EPC (certificaciones, avance, presupuesto) guía la construcción de variables a partir del histórico del portal anterior y de las planillas de control, y las predicciones se integran al ERP mediante su propio tablero, de modo que llegan al jefe de proyecto en la misma herramienta en que gestiona su trabajo.

2.4.2 ANALÍTICA PREDICTIVA EN GESTIÓN DE PROYECTOS

La técnica de pronóstico clásica en control de proyectos es la gestión del valor ganado (*earned value management*, EVM). **Fleming2010<empty citation>** la presentan como la integración del alcance, el cronograma y el costo contra una línea base única: en cada corte se compara el valor planificado con el valor ganado (el trabajo efectivamente realizado, valorado al presupuesto) y con el costo real, y de esas tres magnitudes se derivan índices de desempeño de costo (CPI) y de cronograma (SPI). Los índices permiten proyectar el resultado final mediante estimaciones al completar (*estimate at completion*, EAC). El EVM es, por tanto, un pronóstico que extrapola el desempeño observado hasta la fecha.

Esa extrapolación tiene límites conocidos. **Vandevoorde2006<empty citation>** recuerdan que el EVM se desarrolló originalmente para gestionar costos y que su empleo para pronosticar la duración ha sido escaso; comparan los indicadores clásicos de cronograma con los del cronograma ganado (*earned schedule*, ES) y contrastan tres métodos de pronóstico de duración sobre un ejemplo y sobre datos reales. Su trabajo documenta que el indicador SPI clásico, medido en unidades monetarias, se degrada como señal hacia el final del proyecto, lo que motivó la variante ES medida en tiempo. Para uno de los tres casos de este trabajo, el retraso en la fecha de cierre, ese antecedente es relevante: el problema no es nuevo, y las técnicas tradicionales lo resuelven peor que el costo.

La literatura reciente propone reemplazar la extrapolación de índices por modelos aprendidos del histórico. **Wauters2016<empty citation>** comparan cinco métodos de inteligencia artificial

para predecir la duración final de un proyecto y encuentran que superan a los mejores métodos EVM/ES, sobre todo en las etapas temprana y media, a condición de que los conjuntos de entrenamiento y de prueba sean al menos similares; al aumentar la discrepancia entre ambos, los métodos de IA pierden ventaja. En el frente del costo, İnan et al. (2022) construyen un modelo de red neuronal recurrente (LSTM) alimentado con siete características que incluyen factores de cronograma y desempeño y sus medias móviles, y reportan, sobre 300 experimentos, estimaciones más precisas que el modelo tradicional basado en índices del EVM. Ottaviani et al. (2025) generalizan el enfoque con un pipeline automatizado de balanceo, mejora de características, entrenamiento y evaluación, probado con 30 algoritmos sobre 50 proyectos reales de construcción; sus modelos superan al EVM y a la gestión de cronograma convencionales en exactitud, precisión y oportunidad, y advierten que la literatura previa descuidó el sobreajuste.

Tres consecuencias de esa evidencia orientan el diseño del módulo predictivo. Primera, la condición de similitud señalada por **Wauters2016**<empty citation> es el argumento técnico para que la capa Silver del pipeline de datos mapee el esquema histórico del portal anterior (MySQL) y el del ERP nuevo (PostgreSQL) a las mismas variables: un modelo entrenado sobre proyectos cerrados solo puede predecir sobre proyectos activos si ambos se describen del mismo modo. Segunda, los indicadores de desempeño y sus medias móviles, empleados como características por İnan et al. (2022), son candidatos naturales a variables predictoras en la medida en que el histórico permita reconstruirlos, lo que se determina en la exploración de datos del Capítulo III. Tercera, la advertencia sobre el sobreajuste de Ottaviani et al. (2025) refuerza el protocolo de evaluación con conjunto de prueba independiente y validación cruzada ya adoptado en la sección 2.2.

2.4.3 PREDICCIÓN DE DESVIACIONES Y COSTOS

Las desviaciones de costo y de plazo no son un fenómeno aleatorio, sino sistemático. **Flyvbjerg2002**<empty citation>, sobre una muestra de 258 proyectos de infraestructura de transporte por 90 mil millones de dólares, concluyen que las estimaciones usadas para decidir su ejecución son sistemáticamente engañosas y que la subestimación no se explica por error, sino por tergiversación estratégica. Aunque se trata de obra pública y no de proyectos EPC, el

hallazgo sostiene la premisa de este trabajo: si el sobre costo tiene un patrón, puede aprenderse del histórico. En el sector EPC, Habibi et al. (2018) señalan que más de la mitad de los proyectos enfrenta retrasos significativos y escaladas de costo importantes, y revisan los indicadores líderes de desempeño de costo y cronograma, con poca consistencia entre ellos y una concentración de los estudios en la fase de construcción. La noción de indicador líder es la que el módulo predictivo operacionaliza como variable predictora.

Sobre qué técnica emplear, las revisiones del área convergen en los métodos basados en árboles. **Elmousalami2020<empty citation>** revisa las técnicas de inteligencia artificial para la estimación paramétrica de costos en construcción, desde la lógica difusa y las redes neuronales hasta los árboles de decisión y el *gradient boosting*, y en un caso de estudio con veinte técnicas obtiene la mejor exactitud con XGBoost (MAPE de 9,09 % y R^2 ajustado de 0,929), resultado que fue objeto de una discusión posterior en la misma revista. Destaca, además, el manejo de valores faltantes y atípicos de estos métodos, cuestión pertinente para un histórico afectado por los defectos de captura del ERP. Shamim et al. (2025), en una revisión sistemática de 39 artículos (2016–2024), encuentran que las redes neuronales son la técnica más frecuente (26,3 % de los estudios) y que los árboles de decisión y el *gradient boosting* ofrecen mejoras sustanciales, mientras que la regresión conviene cuando las relaciones son lineales; los rangos de exactitud que reportan agregan métricas heterogéneas, por lo que se toman como tendencia y no como referencia comparable.

Dos trabajos recientes ilustran la arquitectura de solución que este proyecto adopta. Nguyen et al. (2026) combinan XGBoost con valores SHAP para predecir sobre costos a partir de variables como tamaño del proyecto, costo estimado, presión de cronograma y cambios de diseño, con validación cruzada de cinco particiones y evaluación con MAE y R^2 ; su conjunto de datos, sin embargo, consta de mil observaciones simuladas, por lo que se cita como ejemplo de método y no como evidencia de desempeño. Shen et al. (2025) parten de que los modelos basados solo en datos no incorporan el conocimiento experto del dominio y proponen, para predecir retrasos, un marco sobre XGBoost con características derivadas de reglas, penalizaciones en la función de pérdida y reglas de postprocesamiento, que sobre datos reales mejora la exactitud y reduce los falsos negativos. El cuadro 2.4 resume los estudios revisados.

Cuadro 2.4. Estudios revisados sobre predicción de desviaciones, costos y plazos en proyectos

Estudio	Técnica	Variable	Dato relevante
Vandevoorde2006<empty citation>	Aplicación EVM y ES	Duración	El EVM se creó para costos y se usó poco para pronosticar duración; compara tres métodos sobre datos reales
Wauters2016<empty citation>	Diferentes métodos de IA	Duración	Superan a EVM/ES si entrenamiento y prueba son similares; pierden ventaja al crecer la discrepancia
İnan et al. (2022)	LSTM	Costo	Siete características con medias móviles de indicadores; 300 experimentos; más preciso que el EVM por índices
Ottaviani et al. (2025)	Pipeline con 30 algoritmos	EAC	50 proyectos reales de construcción; supera a EVM y gestión de cronograma; alerta sobre sobreajuste
Habibi et al. (2018)	Revisión de indicadores líderes	Costo y plazo (EPC)	Más de la mitad de los proyectos con retrasos y escaladas de costo importantes
Elmousalami2020<empty citation>	Modelos de IA	Costo	XGBoost con la mejor exactitud: MAPE 9,09 %, R^2 ajustado 0,929
Shamim et al. (2025)	Revisión sistemática (39 artículos)	Costo	ANN más frecuente (26,3 %); árboles y <i>gradient boosting</i> con mejoras sustanciales
Nguyen et al. (2026)	XGBoost + SHAP	Sobrecosto	CV de 5 particiones, MAE y R^2 ; 1 000 observaciones simuladas
Shen et al. (2025)	XGBoost con reglas de dominio	Retraso	Mejora la exactitud y reduce falsos negativos sobre datos reales

Fuente: Elaboración propia a partir de las fuentes citadas.

De esta revisión se desprende el planteo de los tres casos del proyecto: la desviación en certificaciones como problema de clasificación, y el sobrecosto y el retraso en la fecha de cierre como problemas de regresión, los tres con *gradient boosting* sobre el histórico del portal anterior y las planillas de control. Ninguna de las fuentes revisadas aporta cifras de sobrecosto para el sector de petróleo, gas y minería en Bolivia, por lo que la magnitud de las desviaciones en ISI Mustang solo puede establecerse con sus propios datos. Se prevé incorporar reglas de negocio como variables y validaciones de consistencia, como en Shen et al. (2025), y acompañar cada predicción con los factores que más pesaron en ella. Las predicciones se muestran únicamente en el tablero del ERP; los reportes de Power BI se mantienen en el nivel descriptivo.

2.5 GESTIÓN DE PROYECTOS EPC

2.5.1 DEFINICIÓN Y CARACTERÍSTICAS

Un proyecto EPC (*engineering, procurement and construction*) es aquel en el que un mismo contratista asume, bajo un único contrato, la ingeniería de diseño, la procura de equipos y materiales y la construcción de la instalación. La forma contractual de referencia es el Libro Plata de **FIDIC2017**<empty citation>, las condiciones de contrato para proyectos EPC «llave en mano», en las que el contratista responde de manera integral por el diseño y la ejecución y entrega al cliente una instalación lista para operar. **Galloway2009**<empty citation> describe la posición del contratista desde el punto de vista del propietario: una ventanilla única y a la vez la última parada para todos los costos del proyecto, desde su inicio hasta el cierre.

El rasgo que distingue esta modalidad es la distribución del riesgo. La propia FIDIC declara que el Libro Plata no es adecuado cuando el comitente pretende supervisar de cerca la obra ni cuando existe obra subterránea sustancial que los oferentes no pueden inspeccionar (**FIDIC2017**); en otras palabras, es la forma contractual en la que más riesgo se traslada al contratista, y a cambio el cliente obtiene un precio y un plazo comprometidos con menor intervención de su parte. **Galloway2009**<empty citation> matiza el atractivo de ese arreglo: aunque el esquema puede ahorrar al propietario sumas importantes en el precio inicial y en órdenes de cambio, no es la

solución definitiva que se percibe, porque el tratamiento del cambio sigue siendo el problema peor definido en ese entorno. Para el contratista, cada cambio de alcance que no logra trasladar al precio es una desviación que absorbe.

Las tres fases que dan nombre a la modalidad forman una cadena. **Yeo2002<empty citation>** proponen para los proyectos EPC un marco de procura que integra la gestión de la cadena de suministro con la cadena crítica derivada de la teoría de restricciones, lo que sitúa a la procura, con sus plazos de entrega y sus dependencias con proveedores y subcontratistas, como fuente principal de incertidumbre entre la ingeniería y la construcción. En los sectores donde esta modalidad predomina los resultados son con frecuencia adversos: **Merrow2011<empty citation>**, a partir de plataformas costa afuera, plantas químicas y proyectos de petróleo, gas y minerales, sostiene que más de la mitad de los grandes proyectos de ingeniería y construcción tienen resultados pobres, con sobrecostos y retrasos, y lo atribuye a una gestión inadecuada, a la disfunción de los equipos, a una débil rendición de cuentas y a una inversión técnica insuficiente en las etapas tempranas.

ISI Mustang ejecuta proyectos de este tipo en gas, petróleo, minería y energía, y ocupa en ellos la posición del contratista que describe la literatura: asume la ingeniería, la procura y la construcción, y con ellas el riesgo de que el costo o el plazo se aparten de lo comprometido. En su ERP cada proyecto se gestiona como un centro de costo con presupuesto, moneda, responsable y vigencia propios, sobre el que se imputan horas, compras y viáticos. Esa correspondencia entre la unidad contractual y la unidad de registro es lo que hace posible observar, proyecto por proyecto, el fenómeno que las dos subsecciones siguientes describen: cómo se mide el avance y cómo se manifiestan las desviaciones.

2.5.2 CERTIFICACIONES Y CONTROL DE AVANCE

El marco de referencia para el control de avance es la gestión del valor ganado (*earned value management*, EVM). **Anbari2003<empty citation>** la define como un método que integra el alcance, el costo y el tiempo, y que requiere el monitoreo periódico de los gastos reales y de los logros físicos del alcance; a partir de ellos calcula variaciones de costo y de cronograma

junto con índices de desempeño, pronostica el costo a la terminación y señala la posible necesidad de acciones correctivas. El **PMI2019EVM** lo formula en los mismos términos en su norma vigente: una metodología para integrar alcance, cronograma y recursos, medir de forma objetiva el desempeño y el avance, y pronosticar el resultado del proyecto. **Fleming2010** describen el ciclo de aplicación: definir el alcance mediante una estructura de desglose del trabajo, fijar una línea base de medición del desempeño, medir periódicamente el valor ganado, calcular variaciones e índices y pronosticar el costo final.

El método se apoya en tres magnitudes acumuladas a una fecha de corte: el valor planificado (PV), presupuesto del trabajo que debía haberse hecho; el valor ganado (EV), presupuesto del trabajo efectivamente realizado; y el costo real (AC), lo gastado en ese trabajo. De su comparación derivan las variaciones e índices del cuadro 2.5, que la séptima edición de la guía PMBOK incluye entre las métricas del dominio de desempeño de medición (**PMI2021**). La dificultad práctica está en el valor ganado, porque exige medir el avance físico; la norma admite para ello varios métodos, entre ellos la fórmula fija, los hitos ponderados, el porcentaje completado y la medición física (**PMI2019EVM**).

Cuadro 2.5. Variaciones, índices y pronóstico de la gestión del valor ganado

Indicador	Cálculo	Lectura
Variación de costo (CV)	$EV - AC$	Negativa: el trabajo realizado costó más de lo presupuestado
Variación de cronograma (SV)	$EV - PV$	Negativa: se hizo menos trabajo del planificado a la fecha
Índice de desempeño de costo (CPI)	EV/AC	Menor que 1: sobre costo en lo ejecutado
Índice de desempeño de cronograma (SPI)	EV/PV	Menor que 1: atraso respecto del plan
Estimación a la terminación (EAC)	BAC/CPI (forma básica)	Costo final proyectado si se mantiene el desempeño actual; BAC es el presupuesto total

Fuente: Elaboración propia a partir de Anbari2003, Fleming2010 y PMI2019EVM.

En un contrato EPC el avance es la base del pago. La cláusula de precio y pago del Libro Plata establece que el contratista presenta estados de cuenta periódicos con el monto que considera devengado y que el propio comitente los evalúa y paga, sin el ingeniero independiente que en otras formas de la suite emite el certificado de pago provisional (**FIDIC2017**). La certificación de avance es así el acto contractual por el cual el cliente reconoce como ejecutada una porción del alcance y habilita su facturación. En el ERP de ISI Mustang se registra como certificación, el ingreso ejecutado de un centro de costo, que el gerente del centro solicita y el área de compras registra y cobra, con estados de borrador, enviada y registrada. En la interpretación que adopta este trabajo, esa entidad cumple la función del ciclo de estado de cuenta, certificación y pago de FIDIC: cada certificación es a la vez una medición de lo ejecutado y un ingreso reconocido.

Los índices del valor ganado se usan además como predictores. **Lipke2009<empty citation>** señalan que el método ofrece desde su origen procedimientos para pronosticar el costo final pero no la duración, e integran el valor ganado, el cronograma ganado (*earned schedule*) y límites de confianza estadísticos para derivar el rango de resultados probables del costo y la duración finales. Ese uso es el antecedente directo del enfoque de este proyecto, con una precisión de alcance: el ERP de la empresa no gestiona una estructura de desglose del trabajo, por lo que no aplica EVM en sentido estricto; lo que ofrece, por centro de costo, es un presupuesto con línea base y seguimiento, los costos imputados y la serie de certificaciones, con los que se prevé aproximar las variaciones e índices del cuadro anterior como variables del modelo, según el análisis de datos del Capítulo III.

2.5.3 DESVIACIONES PRESUPUESTARIAS Y DE CRONOGRAMA

Los sobre costos y los retrasos en grandes proyectos son sistemáticos y no excepcionales. **Flyvbjerg2014<empty citation>** lo resume en la «ley de hierro» de los megaproyectos: por encima del presupuesto, por encima del plazo, una y otra vez; estima el gasto mundial en ellos entre seis y nueve billones de dólares anuales, cerca del ocho por ciento del producto bruto mundial, y explica la persistencia del patrón por un modelo de romper y arreglar, en el que los proyectos se corrigen sobre la marcha. En la modalidad EPC la evidencia coincide: Habibi et al. (2018) señalan que más de la mitad de los proyectos enfrenta retrasos significativos y escaladas de costo importantes, y

en hidrocarburos **Olaniran2015<empty citation>** recogen un reporte según el cual el 64 % de los megaproyectos en ejecución en el mundo presenta sobrecostos, y advierten que la investigación sobre por qué y cómo ocurren es limitada.

Las causas identificadas se repiten entre estudios. **Assaf2006<empty citation>**, en una encuesta en Arabia Saudita, identifican 73 causas de retraso, encuentran que el 70 % de los proyectos considerados sufrió sobreplazo y que la causa más frecuente para las tres partes es la orden de cambio. **Olawale2010<empty citation>** distinguen las causas de las desviaciones de los factores que impiden controlarlas: en 250 organizaciones del Reino Unido, los cinco inhibidores principales del control de costo y plazo son los cambios de diseño, los riesgos e incertidumbres, la evaluación inexacta de la duración, la complejidad y el incumplimiento de los subcontratistas; sus medidas de mitigación se clasifican en preventivas, predictivas, correctivas y organizacionales. Para EPC, Habibi et al. (2018) revisan más de doscientos artículos en busca de indicadores adelantados de costo y plazo por fase de ingeniería, procura y construcción; confirman el cambio de diseño como causa principal, agregan la escasez de recursos y la fluctuación de precios, y observan que todo retraso termina generando sobrecosto.

En el sector de ISI Mustang los estudios son menos numerosos pero apuntan en la misma dirección. **Olaniran2015<empty citation>** sostienen que los sobrecostos en petróleo y gas surgen de interacciones complejas entre las características del proyecto, las personas, la tecnología, la estructura y la cultura, y proponen la teoría del caos como marco para explicarlas. Un estudio preliminar en fabricación EPC para petróleo y gas en Malasia, basado en entrevistas a diez especialistas, vincula los sobrecostos con la asignación de recursos y ubica en el presupuesto insuficiente el problema principal (Mohd Roshdi et al., 2022); su alcance es limitado. El rasgo común de estos trabajos, y de **Merrow2011<empty citation>**, es que la desviación no obedece a una causa aislada sino a la combinación de muchas, lo que argumenta a favor de un modelo que integre múltiples variables en lugar de una regla de umbral sobre un solo indicador.

Para este proyecto, la literatura reseñada delimita qué se entiende por desviación. La desviación presupuestaria es la diferencia entre el costo real de un proyecto y su presupuesto, que en la nomenclatura del valor ganado corresponde a la variación y al índice de costo; la desviación de cronograma es la diferencia entre la fecha de cierre efectiva y la comprometida; y la desviación

en certificaciones es la diferencia entre lo certificado a una fecha y lo planificado certificar, la forma en que el atraso o el sobrecosto se hacen visibles en el ingreso. Son las tres respuestas que el módulo predictivo busca anticipar. Los indicadores adelantados de Habibi et al. (2018), las órdenes de cambio de **Assaf2006<empty citation>** y los inhibidores de **Olawale2010<empty citation>** orientan la búsqueda de variables predictoras en el histórico; la categoría de medidas predictivas de estos últimos autores es el lugar que ocupa un módulo de predicción integrado al ERP.

2.6 INGENIERÍA DE SOFTWARE

2.6.1 PROCESO DE DESARROLLO DE SOFTWARE

Un proceso de desarrollo de software es el conjunto de actividades que conduce a la producción de un producto de software. La norma **ISO12207-2017<empty citation>** establece un marco común de procesos del ciclo de vida, con terminología definida, que contiene procesos, actividades y tareas aplicables durante la adquisición, el suministro, el desarrollo, la operación, el mantenimiento y el retiro de sistemas, productos y servicios de software; los agrupa en procesos de acuerdo, de habilitación organizacional, de gestión técnica y técnicos. La norma aclara, sin embargo, que no prescribe ningún modelo de ciclo de vida ni metodología de desarrollo: quien la usa es responsable de elegir un modelo para su proyecto y de mapear sobre él los procesos y actividades. Esa distinción entre proceso (qué actividades se realizan) y modelo de ciclo de vida (cómo se ordenan en el tiempo) es la que organiza esta sección y la siguiente.

Sommerville2016<empty citation> presenta los modelos de proceso como representaciones abstractas y distingue tres: el modelo en cascada, dirigido por plan, con fases separadas de especificación y desarrollo; el desarrollo incremental, que intercala especificación, desarrollo y validación; y la integración y configuración, basada en reutilizar componentes existentes. **Pressman2020<empty citation>** agrupan los primeros bajo la etiqueta de modelos prescriptivos, en contraste con los procesos ágiles. El modelo en cascada suele remontarse a **Royce1970<empty citation>**, que propone una secuencia de requisitos de sistema, requisitos de software, análisis, diseño de programa, codificación, pruebas y operación, aunque el nombre es

posterior y el propio autor advierte que esa implementación es riesgosa e invita al fracaso, porque la fase de pruebas al final es el primer momento en que se experimentan, y no solo se analizan, los tiempos y el almacenamiento, y un fallo ahí obliga a rediseñar. Sus correcciones incluyen documentar el diseño, hacerlo dos veces (que la versión entregada sea en realidad la segunda) e involucrar al cliente.

Esa crítica se formaliza en el modelo en espiral. **Boehm1988<empty citation>** lo presenta como un enfoque evolutivo guiado por el riesgo que provee un marco para conducir el proceso de software; según **Larman2003<empty citation>**, su aporte fue hacer prominente el concepto de iteraciones guiadas por riesgo, con un paso explícito de evaluación de riesgos en cada iteración. Los métodos ágiles llevan la iteración más lejos: el manifiesto de **Beck2001<empty citation>** antepone los individuos y sus interacciones a los procesos y herramientas, el software funcionando a la documentación exhaustiva, la colaboración con el cliente a la negociación contractual y la respuesta al cambio al seguimiento de un plan. **Larman2003<empty citation>** señalan que los diecisiete firmantes representaban métodos como XP, Scrum y DSDM, todos ellos iterativos e incrementales. El cuadro 2.6 resume los modelos comparados.

Cuadro 2.6. Modelos de proceso de software comparados

Modelo	Rasgo central	Fortaleza	Limitación
Cascada	Fases secuenciales con especificación cerrada antes del desarrollo	Simple de explicar; proceso visible y medible	Los problemas se descubren en las pruebas finales; el cambio es costoso
Espiral	Ciclos guiados por el riesgo, con evaluación explícita en cada uno	Reduce el riesgo antes de comprometer recursos	Exige experiencia en análisis de riesgos
Incremental	Especificación, desarrollo y validación intercalados en entregas parciales	Menor costo del cambio; retroalimentación temprana; entrega útil más rápida	Menor visibilidad para la gestión; la estructura se degrada sin refactorización
Ágil	Iteraciones cortas, colaboración continua con el cliente, respuesta al cambio	Adaptación rápida; software funcionando como medida de avance	Depende de la disponibilidad del cliente y de equipos autoorganizados

Fuente: Elaboración propia a partir de Royce1970<empty citation>, Boehm1988<empty citation>, Sommerville2016<empty citation>, Larman2003<empty citation> y Beck2001<empty citation>.

Para este trabajo, las actividades del proceso quedan fijas: cada incremento comprende definición de alcance, análisis, diseño, desarrollo y validación, que corresponden a los procesos técnicos de la norma. Lo que se elige es el modelo de ciclo de vida que las ordena, y se prevé que sea el iterativo e incremental, por las razones que se desarrollan en la subsección siguiente.

2.6.2 METODOLOGÍA DE DESARROLLO ITERATIVA E INCREMENTAL

Larman2003<empty citation> reconstruyen la historia del desarrollo iterativo e incremental y muestran que, aunque muchos lo consideran una práctica moderna, su aplicación se

remonta a mediados de los años cincuenta, con un ejemplo temprano en el Proyecto Mercury de la NASA, que trabajó con iteraciones de medio día. La definición clásica proviene de **Basili1975<empty citation>**, citados por **Larman2003<empty citation>**: comenzar con una implementación simple de un subconjunto de los requisitos y mejorar iterativamente la secuencia de versiones hasta implementar el sistema completo, incorporando en cada iteración modificaciones de diseño junto con nuevas capacidades funcionales. El rasgo común de todos los enfoques de esta familia es evitar el paso único, secuencial, dirigido por documentos y con compuertas entre fases; y la palabra «iterativo» implica no solo revisar el trabajo, sino avanzar evolutivamente.

La misma fuente documenta proyectos que fijaron de antemano el número de incrementos: el sistema de defensa de TRW para el ejército estadounidense se desarrolló en cinco iteraciones en 1972; y el proyecto Trident de IBM, en cuatro iteraciones de unos seis meses. En el plano normativo, la norma militar Mil-Std-498 de 1994 reemplazó la exigencia de cascada de una sola pasada por un desarrollo en una o más construcciones incrementales, cada una de las cuales implementa un subconjunto especificado de las capacidades planificadas. **Sommerville2016<empty citation>** resume los beneficios del desarrollo incremental frente a la cascada: menor costo de acomodar cambios en los requisitos, retroalimentación más fácil del cliente y entrega más rápida de software útil; y sus problemas: menor visibilidad para la gestión y degradación de la estructura si no se refactoriza.

Tres condiciones del proyecto justifican la elección del modelo iterativo e incremental sobre la cascada. La primera es que el desarrollo lo realiza una sola persona: un modelo secuencial concentraría toda la validación al final, que es exactamente el riesgo que **Royce1970<empty citation>** describe. La segunda es que el alcance se fija por incrementos: cada uno implementa un subconjunto especificado de las capacidades planificadas, en la lógica de las construcciones incrementales de Mil-Std-498, y cierra con una validación propia, de modo que al terminar cada uno existe software funcionando. La tercera es la dependencia de los datos: las variables predictoras y los casos de uso solo se confirman al explorar el histórico del portal anterior, lo que impide cerrar la especificación completa de antemano; el modelo incremental es el que abarata ese cambio. Como la norma **ISO12207-2017<empty citation>** deja la elección del modelo de ciclo de vida al proyecto, esta decisión no contradice el marco de procesos.

Se prevé, en consecuencia, un desarrollo en cinco incrementos: los procesos del portal anterior migrados al ERP, la autenticación centralizada con Keycloak, el pipeline de datos y el entrenamiento de los modelos, el microservicio de predicción con su integración al ERP y al tablero, y la integración con Power BI y MCP. La documentación de cada incremento en el Capítulo IV mitiga el problema de visibilidad que señala Sommerville; la degradación de la estructura se atiende con la arquitectura por capas que se describe a continuación.

2.6.3 ARQUITECTURA DE SOFTWARE POR CAPAS

Bass2021<empty citation> entienden la arquitectura de software como el conjunto de estructuras necesarias para razonar sobre el sistema, compuestas por elementos, relaciones entre ellos y propiedades de ambos, y distinguen las estructuras de módulos, entre ellas las capas, de las de componentes y conectores y de las de asignación al entorno de despliegue. **Richards2020<empty citation>** añaden que las decisiones de arquitectura definen las reglas de cómo debe construirse el sistema: por ejemplo, que solo las capas de negocio y de servicios puedan acceder a la base de datos, con lo que se impide que la capa de presentación haga llamadas directas.

El patrón en capas es una estructura de módulos. **Sommerville2016<empty citation>** lo describe como la organización del sistema en capas, cada una con funcionalidad relacionada, donde cada capa provee servicios a la capa superior y las inferiores representan los servicios centrales; su ventaja principal es que permite reemplazar una capa entera mientras se mantenga su interfaz, y sus desventajas son la dificultad de lograr una separación limpia y el costo de rendimiento de atravesar varios niveles para atender una petición. **Richards2020<empty citation>** formulan el principio de capas de aislamiento, según el cual una capa depende solo de la inmediata inferior. **Fowler2002<empty citation>** fija el vocabulario para aplicaciones empresariales con tres capas principales, presentación, lógica de dominio y fuente de datos, y señala que esas capas lógicas pueden ejecutarse en distintos nodos físicos.

Cuando la separación se lleva más allá de un proceso, se llega a los microservicios. **Lewis2014<empty citation>** los definen como un enfoque para desarrollar una única aplicación

como un conjunto de servicios pequeños, cada uno ejecutándose en su propio proceso y comunicándose mediante mecanismos ligeros, a menudo una API HTTP de recursos, y entre sus características cuentan la organización alrededor de capacidades de negocio, la gestión descentralizada de datos y la gobernanza descentralizada, que admite distintas tecnologías por servicio. **Newman2021<empty citation>** trata la extracción de capacidades de un monolito hacia servicios, la comunicación síncrona por HTTP entre ellos y la autenticación entre servicios. **Richards2020<empty citation>** ponen capas y microservicios en el mismo marco comparativo: el estilo distribuido aporta contexto acotado, aislamiento de datos y una capa de API, a cambio de costos operativos.

En este trabajo se combinan ambos niveles. Dentro de cada servicio se aplica el patrón en capas: en el ERP, controladores, servicios de dominio y acceso a datos con Prisma sobre PostgreSQL, y Angular como presentación; en el módulo predictivo, rutas, lógica de predicción y acceso a modelos y datos. Entre servicios, el módulo predictivo se construye como un microservicio en FastAPI separado del ERP en NestJS: es una decisión de diseño justificada porque corre en su propio proceso y expone una API HTTP, es dueño de sus modelos y de sus datos analíticos, usa un lenguaje distinto al del ERP en la lógica de la gobernanza descentralizada, y puede reentrenarse y desplegarse sin tocar el ERP, lo que responde a la modificabilidad y la integrabilidad que **Bass2021<empty citation>** tratan como atributos de calidad. Todos los servicios validan la identidad contra Keycloak y se despliegan en contenedores Docker en un servidor propio de la empresa. Se prevé que esta organización se concrete en la arquitectura de seis capas del módulo predictivo, cuyo diseño detallado corresponde a la sección 3.4.

2.7 TECNOLOGÍAS DEL STACK DE DESARROLLO

Esta sección presenta las tecnologías con las que se construye el sistema y justifica su elección. Conviene distinguir dos grupos. El stack del ERP (NestJS, Prisma, PostgreSQL y Angular) está fijado por el sistema existente y este trabajo lo toma como dado. El stack del módulo predictivo (Python, DuckDB, Polars, Scikit-learn, XGBoost y FastAPI) es una decisión de diseño de este proyecto, todavía no implementada, y es la que requiere justificación. Keycloak, el servidor MCP

y Power BI se integran con ambos grupos y se tratan en las secciones 2.8 a 2.10.

2.7.1 PYTHON Y SU ECOSISTEMA ML

Python es un lenguaje de programación interpretado, interactivo y orientado a objetos, que admite además los paradigmas procedural y funcional, es portable entre Linux, macOS y Windows, y es extensible en C o C++ (**PythonFAQ**). Esta última propiedad explica la forma que tomó su ecosistema científico: las bibliotecas numéricas exponen una interfaz en Python sobre un núcleo compilado. La base de ese ecosistema es NumPy, que **Harris2020<empty citation>** presentan como la biblioteca principal de programación con arreglos del lenguaje y como el fundamento sobre el que se construye el resto de las bibliotecas científicas, con el papel adicional de capa de interoperabilidad entre ellas. La programación con arreglos ofrece una sintaxis compacta para operar sobre vectores y matrices completos en lugar de elemento por elemento; ese estilo vectorizado reaparece, con otra implementación, en los motores de DuckDB y Polars.

Sobre NumPy se apoya pandas, presentada por **McKinney2010<empty citation>** como una biblioteca de estructuras de datos para el cómputo estadístico, motivada por los problemas prácticos de trabajar con los conjuntos de datos propios de las finanzas y la estadística. Sus dos estructuras primarias son la *Series*, unidimensional y etiquetada, y el *DataFrame*, tabular y con columnas de tipos heterogéneos; la biblioteca resuelve de forma nativa el manejo de valores faltantes, la alineación por etiquetas, las operaciones de agrupamiento y combinación, y funciones específicas para series temporales como ventanas móviles, desplazamientos y rezagos (**PandasOverview**). En este proyecto pandas cumple dos funciones: la exploración del histórico en el Capítulo III y el puente hacia las bibliotecas de modelado, que aceptan *DataFrame* como entrada.

La capa de modelado la forman Scikit-learn y XGBoost. **Pedregosa2011<empty citation>** presentan Scikit-learn como una biblioteca de aprendizaje automático que pone el énfasis en la facilidad de uso, el desempeño, la documentación y la consistencia de su interfaz, con licencia BSD y dependencias mínimas. XGBoost, cuyo algoritmo se describió en la sección 2.2, se emplea aquí como biblioteca: su paquete para Python incluye estimadores compatibles con la interfaz de

Scikit-learn (XGBClassifier y XGBRegressor), lo que permite usarlos dentro de los *pipelines* y de las rutinas de validación cruzada de esa biblioteca (**Chen2016**; **XGBoostDocs**). La elección de Python para el módulo predictivo responde a que toda la cadena (arreglos, tablas, estimadores y servicio de predicción) se resuelve en un mismo lenguaje, y a que los tres componentes restantes de esa cadena, DuckDB, Polars y FastAPI, se usan desde Python. La versión concreta del intérprete y de cada biblioteca se fija en el Capítulo IV al construir el pipeline.

2.7.2 DUCKDB Y POLARS

La decisión de no ejecutar el análisis sobre la base de datos de producción del ERP tiene un fundamento de arquitectura. **Stonebraker2005**<empty citation> sostienen que la arquitectura tradicional de los sistemas de gestión de bases de datos, diseñada y optimizada para el procesamiento de datos de negocio, se ha usado para sostener aplicaciones con características y requisitos muy distintos, y argumentan que la idea de un motor único para todos los usos ya no es aplicable: el mercado se fragmenta en motores independientes, y el almacén de datos es uno de los dos ejemplos con que sustentan el argumento. Trasladado a este proyecto, el procesamiento transaccional en línea (*online transaction processing*, OLTP) del ERP y el procesamiento analítico en línea (*online analytical processing*, OLAP) del pipeline Medallion tienen requisitos distintos y se resuelven con motores distintos. El argumento es de arquitectura, no de rendimiento medido en este trabajo.

El motor analítico elegido es DuckDB. **Raasveldt2019**<empty citation> lo presentan como un sistema de gestión de datos diseñado para ejecutar consultas SQL analíticas embebido en otro proceso, una necesidad que la popularidad de SQLite hacía evidente y que ningún sistema cubría para cargas analíticas; se distribuye como software libre con licencia permisiva. Según su documentación, no tiene dependencias externas, usa un motor de ejecución columnar y vectorizado, que procesa un lote de valores por operación, y es extensible en tipos, funciones y formatos (**DuckDBWhy**). Dos extensiones justifican su papel en la capa Silver: la de MySQL permite consultar directamente una instancia MySQL en ejecución y cargar sus tablas en DuckDB, y la de PostgreSQL hace lo propio con esa base, ambas mediante la sentencia ATTACH (**DuckDBMySQL**; **DuckDBPostgres**). Un solo motor SQL, embebido en el proceso Python del ETL

y sin servidor adicional que desplegar, lee así el histórico del portal anterior, los proyectos activos del ERP nuevo y las planillas de la capa Bronze. La documentación de la extensión MySQL no indica versiones mínimas soportadas, por lo que su compatibilidad con MySQL 5.7, la versión del portal anterior, se verifica empíricamente en el Capítulo IV.

Para la transformación de la capa Gold se prevé utilizar Polars, una biblioteca de `DataFrame` con núcleo escrito en Rust y disponible para Python (**PolarsDocs**). Su documentación declara ejecución paralela entre los núcleos del procesador, un motor de consultas vectorizado, una interfaz de flujo (*streaming*) para procesar resultados sin tener todos los datos en memoria, y soporte de Apache Arrow. Su interfaz perezosa (*lazy*) difiere la ejecución hasta que se solicita el resultado, lo que le permite filtrar filas y seleccionar columnas lo antes posible durante la lectura (empuje de predicados y de proyecciones); la documentación recomienda preferirla salvo en trabajo exploratorio (**PolarsLazy**). Esas propiedades, y no una comparación de velocidad con pandas que no se ha verificado con fuentes independientes, motivan la elección. El traspaso entre las dos capas es directo: DuckDB consulta de forma nativa los `DataFrame` de Polars presentes en el ámbito de ejecución y devuelve sus resultados como `DataFrame` de Polars mediante la integración con Arrow, sin serializar a disco (**DuckDBPolars**).

2.7.3 FASTAPI COMO MICROSERVICIO

El servicio de predicción se construye sobre el estándar ASGI (*Asynchronous Server Gateway Interface*), que su especificación define como sucesor de WSGI, el estándar de compatibilidad entre servidores web, marcos de trabajo y aplicaciones en Python. Una aplicación WSGI es un único invocable síncrono que recibe una petición y devuelve una respuesta, lo que no admite conexiones de larga duración; ASGI extiende ese contrato al Python asíncrono (**ASGISpec**). Sobre el estándar se apoya Starlette, un marco ligero para construir servicios web asíncronos, con pocas dependencias obligatorias y código completamente anotado con tipos (**StarletteDocs**), y lo ejecuta un servidor ASGI como Uvicorn (**FastAPIASGI**).

La segunda pieza es Pydantic, biblioteca de validación de datos en la que el esquema de validación y serialización se controla mediante anotaciones de tipo; su lógica central está escrita

en Rust, admite un modo estricto sin conversión de tipos y uno laxo con coerción, y emite JSON Schema para integrarse con otras herramientas (**PydanticDocs**). FastAPI combina ambas: según su documentación, se apoya en Starlette para la parte web y en Pydantic para la parte de datos, y está basado en los estándares abiertos OpenAPI y JSON Schema, lo que le permite generar documentación interactiva automática (Swagger UI y ReDoc) y habilitar la generación de clientes en otros lenguajes (**FastAPIDocs**). FastAPI es, en consecuencia, un marco ASGI (**FastAPIASGI**).

Tres razones justifican su elección para el módulo predictivo. Primero, los modelos entrenados con Scikit-learn y XGBoost viven en Python; servirlos desde Python evita reimplementarlos o trasladarlos a otro entorno de ejecución. Segundo, Pydantic define el contrato de entrada y salida del endpoint de predicción (tipos, rangos y valores faltantes admitidos) y FastAPI lo publica como especificación OpenAPI, que es el contrato que el backend NestJS consume para tipar su cliente. Tercero, el modelo asíncrono de ASGI permite que la validación de tokens contra Keycloak (sección 2.8) y las lecturas de la base del ERP no bloqueen al proceso mientras espera la respuesta. El servicio se despliega como un contenedor independiente del ERP, de modo que el modelo puede actualizarse sin redespargar el backend. Las cifras de rendimiento y productividad que la documentación de FastAPI publica sobre sí misma no se toman como evidencia, por tratarse de autoevaluaciones sin estudio independiente.

2.7.4 NESTJS Y ANGULAR

El backend del ERP usa NestJS, un marco para construir aplicaciones de servidor en Node.js que está escrito en TypeScript y lo soporta por completo, combina elementos de la programación orientada a objetos, funcional y reactiva funcional, y utiliza Express como servidor HTTP por defecto (**NestJS Docs**); el ERP corre NestJS 11 sobre Express y Node 22. Su unidad de organización es el módulo, una clase anotada con el decorador `@Module()`: toda aplicación tiene al menos un módulo raíz desde el que Nest construye el grafo de la aplicación, y cada módulo encapsula sus proveedores, de modo que solo pueden inyectarse los propios o los exportados explícitamente por módulos importados (**NestJS Modules**). Esa encapsulación permite agregar al ERP, como módulos separados, la integración con el servicio de predicción, la API para Power BI (sección 2.10) y el servidor MCP (sección 2.9).

El acceso a datos lo resuelve Prisma, un ORM (*object-relational mapping*) para Node.js y TypeScript que ofrece acceso a la base con seguridad de tipos, migraciones y un editor visual de datos; su esquema es la única fuente de verdad de los modelos de base de datos y de aplicación, y las consultas se validan en tiempo de compilación (**PrismaDocs7**). El ERP usa Prisma 7.8, versión en la que el adaptador de controlador es obligatorio para las conexiones directas y la configuración vive en `prisma.config.ts`. La base es PostgreSQL 17, un sistema de gestión de bases de datos objeto-relacional que soporta gran parte del estándar SQL y ofrece integridad transaccional y control de concurrencia multiversión (**PostgreSQLDocs**); esas son las propiedades OLTP que, según el argumento de **Stonebraker2005**<empty citation> recogido en la subsección anterior, conviene no mezclar con la carga analítica.

El frontend es Angular 21.2, un marco web mantenido por un equipo de Google que organiza el código en componentes encapsulados e incorpora un modelo de reactividad de grano fino basado en *signals*, inyección de dependencias, enrutamiento y formularios con validación (**AngularOverview**). El ERP lo usa con renderizado del lado del servidor (*server-side rendering*, SSR), que genera en el servidor una página HTML completa para cada petición y luego la reutiliza en el cliente mediante hidratación (**AngularSSR**). Para una aplicación interna con inicio de sesión, el beneficio relevante no es el posicionamiento en buscadores sino el primer render rápido del dashboard donde se muestran las predicciones. La interfaz se compone con PrimeNG 21.1 y Tailwind 4.1. Todos los componentes se despliegan con Docker, que empaqueta y ejecuta cada aplicación en un contenedor aislado que sirve como unidad de distribución (**DockerDocs**); el monorepo define los servicios en un archivo de Docker Compose, y la portabilidad de las imágenes permite prever la migración desde el servidor propio de la empresa a la nube como trabajo futuro. El cuadro 2.7 resume el stack completo.

Cuadro 2.7. Tecnologías del sistema por capa de la arquitectura

Tecnología	Capa	Rol	Versión
<i>ERP existente</i>			
NestJS (Express, Node)	Backend del ERP	API y módulos de negocio; integraciones con FastAPI, MCP y Power BI	11 (Node 22)
Prisma	Backend del ERP	ORM y migraciones	7.8
PostgreSQL	Base OLTP del ERP	Datos transaccionales; fuente de inferencia	17
Angular (SSR)	Frontend del ERP	Interfaz de usuario y dashboard de predicciones	21.2
PrimeNG / Tailwind	Frontend del ERP	Componentes de interfaz y estilos	21.1 / 4.1
PHP + MySQL	Portal anterior	Fuente del histórico (entrenamiento)	MySQL 5.7
<i>Módulo predictivo (por implementar)</i>			
Python (NumPy, pandas)	Pipeline y modelos	Lenguaje del ETL, exploración y modelado	Cap. IV
DuckDB	Silver	Motor OLAP embebido; lee MySQL, PostgreSQL y archivos	Cap. IV
Polars	Gold	Ingeniería de características	Cap. IV
Scikit-learn / XGBoost	Modelos	Pipelines, validación cruzada y <i>gradient boosting</i>	Cap. IV
FastAPI (Starlette, Pydantic, Uvicorn)	Servicio de predicción	Endpoint de inferencia con contrato OpenAPI	Cap. IV
<i>Transversales</i>			
Keycloak	Identidad	SSO con OAuth 2.0 / OpenID Connect (sección 2.8)	Cap. IV
Servidor MCP	Integración con IA	Tools sobre el ERP y las predicciones (sección 2.9)	Cap. IV
Power BI	Inteligencia de negocios	Reportes descriptivos vía API NestJS (sección 2.10)	—
Docker / Docker	Despliegue	Contenedores en servidor	—

2.8 GESTIÓN DE IDENTIDAD Y ACCESO

2.8.1 OAUTH 2.0 Y OPENID CONNECT

La gestión de identidad y acceso reúne dos procesos distintos: la autenticación, que identifica y valida a un usuario, y la autorización, que le concede acceso (**KeycloakServerAdmin**). El estándar de referencia para el segundo es OAuth 2.0 (*Open Authorization*), un marco que permite a una aplicación obtener acceso limitado a un servicio HTTP, en nombre del propietario de un recurso o por cuenta propia (**RFC6749**). La especificación define cuatro roles: el propietario del recurso (*resource owner*), que cuando es una persona se llama usuario final; el servidor de recursos (*resource server*), que aloja los recursos protegidos y responde a peticiones que presentan tokens de acceso; el cliente, que hace esas peticiones en nombre del propietario; y el servidor de autorización (*authorization server*), que emite los tokens tras autenticar al propietario. Un único servidor de autorización puede emitir tokens aceptados por múltiples servidores de recursos (**RFC6749**), propiedad en la que se apoya el inicio de sesión único (*single sign-on*, SSO) que adopta este proyecto.

OAuth 2.0 distingue clientes confidenciales, que mantienen en secreto sus credenciales porque se ejecutan en un servidor, y clientes públicos, como una aplicación nativa o una aplicación en navegador (**RFC6749**). Sobre esa distinción se organizan sus cuatro concesiones (*grants*). En el código de autorización (*authorization code*) el cliente redirige al usuario al servidor de autorización, que lo autentica y devuelve un código que el cliente canjea por el token; las credenciales del usuario nunca se comparten con el cliente y el token no pasa por el navegador. En el flujo implícito el token se emite directamente al navegador, el servidor no autentica al cliente y el token puede quedar expuesto a otras aplicaciones. En la concesión por credenciales del propietario (*resource owner password credentials*, ROPC) el cliente recibe directamente usuario y contraseña, y la de credenciales de cliente (*client credentials*) se usa cuando el cliente actúa por cuenta propia y está reservada a clientes confidenciales (**RFC6749**).

En un cliente público el código de autorización puede ser interceptado en un canal no protegido por TLS. La prueba de clave para el intercambio de código (*Proof Key for Code*

Exchange, PKCE) mitiga ese ataque: el cliente genera un verificador secreto, envía un derivado en la petición de autorización y presenta el verificador original al canjear el código, de modo que el servidor comprueba que quien canjea es quien inició el flujo (**RFC7636**). La práctica actual de seguridad para OAuth 2.0 (**RFC9700**) lo vuelve obligatorio para los clientes públicos, incluidas las aplicaciones web; desaconseja el flujo implícito; establece que ROPC no debe usarse, porque expone las credenciales del propietario al cliente; y exige que cada servidor de recursos verifique en cada petición que el token fue emitido para él. El borrador de OAuth 2.1 (**OAuth21Draft**), aún en elaboración en el IETF, consolida esa dirección al incorporar PKCE al código de autorización y omitir los flujos implícito y ROPC. **Fett2016**<empty citation> aportan el respaldo académico: en el primer análisis formal extensivo de OAuth 2.0 descubren cuatro ataques hasta entonces desconocidos y demuestran que, con las correcciones propuestas, el protocolo cumple autorización, autenticación e integridad de sesión. La seguridad del marco depende, por tanto, de implementar el conjunto completo de mitigaciones, lo que favorece adoptar un proveedor que ya las incorpora.

El resultado de estos flujos es un token de acceso, una cadena que representa una autorización y que suele ser opaca para el cliente, acompañada opcionalmente de un token de refresco para renovarlo (**RFC6749**). El tipo más usado es el token portador (*bearer*): cualquier parte que lo posea puede usarlo sin demostrar posesión de material criptográfico, por lo que debe protegerse en almacenamiento y en tránsito y enviarse en la cabecera *Authorization* (**RFC6750**). Cuando el token es autocontenido, el formato habitual es el token web JSON (*JSON Web Token*, JWT): una representación compacta de afirmaciones (*claims*) sobre un sujeto, firmada digitalmente, con nombres registrados como emisor (*iss*), sujeto (*sub*), audiencia (*aud*) y expiración (*exp*) (**RFC7519**). OAuth 2.0, sin embargo, no dice quién es el usuario: sin un perfilado adicional es incapaz de proporcionar información sobre la autenticación del usuario final (**OIDCCore2014**). OpenID Connect (OIDC) es la capa de identidad construida sobre OAuth 2.0 que cubre ese vacío: el cliente, llamado parte confiante (*relying party*), solicita el ámbito *openid* a un proveedor de OpenID (OP) y recibe, además del token de acceso, un token de identidad (*ID token*) en formato JWT con afirmaciones sobre la autenticación realizada, que debe validar comprobando emisor y audiencia (**OIDCCore2014**).

El cuadro 2.8 traslada estos roles y flujos al proyecto. El ERP de ISI Mustang autentica hoy con

un endpoint propio que recibe usuario y contraseña y emite un JWT generado por el mismo servidor. Ese esquema sirve para una sola aplicación, pero el proyecto agrega un servicio de predicción en FastAPI, un servidor MCP y una API para Power BI, y **Almeida2022**<empty citation> muestran que en arquitecturas de varios servicios cada uno debe asegurarse individualmente, que el acceso no autorizado a uno puede comprometer el sistema completo, y que OAuth 2.0, OpenID Connect y JWT son los mecanismos documentados para ese escenario. Con un servidor de autorización externo ningún servicio recibe contraseñas, cada uno valida los tokens por firma y audiencia, y la identidad es la misma en todos. Se prevé que Angular opere como cliente público con código de autorización y PKCE, que las llamadas entre servicios usen credenciales de cliente, y que NestJS, FastAPI y el servidor MCP sean servidores de recursos con audiencias distintas.

Cuadro 2.8. Roles y flujos de OAuth 2.0 / OpenID Connect y su uso en el proyecto

Rol o flujo	Definición	En el proyecto
<i>Roles</i>		
Servidor de autorización / OP	Autentica al usuario y emite tokens de acceso e identidad	Keycloak (véase 2.8.2)
Propietario del recurso	Usuario final que concede el acceso	Personal de ISI Mustang que usa el ERP
Cliente público	Aplicación en navegador o nativa, sin secreto	Frontend Angular del ERP
Cliente confidencial	Aplicación en servidor que guarda sus credenciales	NestJS al invocar a FastAPI; Power BI al consumir la API de NestJS; cliente del servidor MCP
Servidor de recursos	Acepta tokens y sirve recursos protegidos	API NestJS, servicio FastAPI y servidor MCP, cada uno con audiencia propia
<i>Flujos</i>		
Código de autorización + PKCE	Inicio de sesión interactivo de personas	Angular ante Keycloak
Credenciales de cliente	Acceso máquina a máquina, solo clientes confidenciales	NestJS hacia FastAPI; Power BI hacia la API de NestJS
Implícito	Token emitido directamente al navegador; desaconsejado	No se usa
ROPC	El cliente recibe usuario y contraseña; no debe usarse	No se usa; sustituye al inicio de sesión propio del ERP

Fuente: Elaboración propia a partir de RFC6749<empty citation>, RFC9700<empty citation> y OIDCCore2014<empty citation>.

2.8.2 KEYCLOAK

Keycloak es una solución de inicio de sesión único para aplicaciones web y servicios

web RESTful cuyo objetivo es simplificar la seguridad: las funciones que los desarrolladores tendrían que escribir por sí mismos vienen incorporadas (**KeycloakServerAdmin**). Es un servidor separado que se administra en la red propia; las aplicaciones se configuran para apuntar a él y ser aseguradas por él mediante protocolos abiertos como OpenID Connect o SAML 2.0. Es un proyecto de código abierto en incubación en la Cloud Native Computing Foundation, y **Thorgersen2023<empty citation>** le dedican una monografía que complementa la documentación oficial. En los términos de la sección anterior, Keycloak es a la vez servidor de autorización de OAuth 2.0 y proveedor de OpenID Connect: expone los endpoints de autorización, token, información de usuario, certificados e introspección, que las aplicaciones descubren a partir de un documento de configuración publicado por cada dominio (**KeycloakSecuringApps**).

Su modelo se organiza en pocos conceptos (**KeycloakServerAdmin**). Un dominio (*realm*) gestiona un conjunto de usuarios, credenciales, roles y grupos; los usuarios inician sesión en un dominio y los dominios están aislados entre sí. Los clientes son las entidades que pueden pedir a Keycloak que autentique a un usuario: aplicaciones que quieren asegurarse y ofrecer inicio de sesión único, o servicios que solo necesitan un token de acceso para invocar a otros servicios protegidos. Los roles identifican una categoría de usuario (administrador, gerente, empleado) y las aplicaciones asignan permisos a roles antes que a usuarios individuales; la asignación de roles puede encapsularse en los tokens, de modo que cada aplicación decide los permisos sobre sus recursos sin consultar al servidor. Keycloak es además un proveedor de identidad (*identity provider*, IdP) capaz de federar la autenticación hacia otros proveedores OIDC o SAML y hacia directorios LDAP, y soporta cualquier plataforma que cuente con una librería de parte confiante de OpenID Connect; **Thorgersen2023<empty citation>** confirman que los adaptadores propios de versiones anteriores fueron reemplazados por librerías estándar.

El aspecto más relevante para una arquitectura de varios servicios es cómo se validan los tokens. Los tokens de acceso de Keycloak son JWT firmados con JSON Web Signature, por lo que pueden validarse localmente con la clave pública del dominio emisor, obtenida del endpoint de certificados y almacenada en caché; consultar el endpoint de introspección en cada petición, en cambio, puede ser lento y sobrecargar el servidor (**KeycloakSecuringApps**). Es el patrón que **Thorgersen2023<empty citation>** describen en su aplicación de ejemplo: el token de identidad identifica al usuario, el token de acceso transporta los permisos para invocar el backend, y este

valida los tokens y controla el acceso por los roles que lee de ellos. Keycloak implementa el código de autorización, con el que la aplicación obtiene token de acceso, de refresco y de identidad, y las credenciales de cliente, que no forman parte de OpenID Connect sino de OAuth 2.0; lista los flujos implícito y ROPC como desaconsejados y recomienda registrar las URI de redirección lo más específicas posible, en especial para clientes públicos (**KeycloakSecuringApps**).

Estas propiedades explican por qué el proyecto reemplaza la autenticación propia del ERP por Keycloak. Con el esquema actual la identidad vive dentro del backend en NestJS: cualquier servicio nuevo que quisiera reconocer al mismo usuario tendría que compartir el secreto con el que se firman los tokens o delegar la validación en el ERP, con el acoplamiento y el riesgo de compromiso en cadena que **Almeida2022**^{<empty citation>} señalan. Con Keycloak, un único dominio contiene a los usuarios y roles de ISI Mustang; Angular autentica a las personas con código de autorización y PKCE; NestJS, FastAPI y el servidor MCP validan localmente el mismo JWT con la clave pública del dominio, comprueban su audiencia y autorizan por los roles que trae; y las llamadas entre servicios, y desde Power BI hacia la API de NestJS, usan credenciales de cliente. Se prevé desplegar Keycloak en contenedor junto al resto de la plataforma, en el servidor propio de la empresa; las librerías con las que cada servicio valida los tokens y la configuración del dominio son decisiones de implementación que se documentan en el Capítulo IV.

2.9 MODEL CONTEXT PROTOCOL (MCP)

2.9.1 DEFINICIÓN Y ARQUITECTURA

El *Model Context Protocol* (MCP) es un protocolo abierto que permite la integración entre aplicaciones basadas en modelos de lenguaje de gran tamaño (*large language models*, LLM) y fuentes de datos y herramientas externas, y provee una forma estandarizada de conectar esos modelos con el contexto que necesitan (**MCPSpec2026**). **Anthropic2024MCP**^{<empty citation>} lo liberó en noviembre de 2024 con una motivación concreta: aun los modelos más sofisticados quedan aislados de los datos, tras silos de información y sistemas heredados, y cada nueva fuente exigía una implementación a medida. **Hou2025**^{<empty citation>} lo caracterizan como un estándar

unificado, bidireccional y con descubrimiento dinámico, distinto de las integraciones manuales de API y de los *plugins* unidireccionales por ser abierto e independiente de la plataforma. En diciembre de 2025 Anthropic donó el protocolo a la Agentic AI Foundation, un fondo dirigido bajo la Linux Foundation, para garantizar su neutralidad respecto de proveedores (**MCPBlog2025AAIF**).

La arquitectura distingue tres participantes: los *hosts*, aplicaciones LLM que inician las conexiones; los clientes, conectores dentro del host; y los servidores, servicios que proveen contexto y capacidades (**MCPSpec2026**). El host crea y administra los clientes, controla sus permisos y aplica las políticas de seguridad y consentimiento; cada cliente mantiene una conexión aislada con un único servidor, y el servidor, que puede ser un proceso local o un servicio remoto, opera con responsabilidades acotadas y no puede leer la conversación completa ni observar a otros servidores (**MCPSpec2025**). Los mensajes se codifican en JSON-RPC 2.0, un protocolo de llamada a procedimiento remoto (*remote procedure call*, RPC) ligero, sin estado e independiente del transporte (**JSONRPC2013**): una solicitud lleva método, parámetros e identificador, y la respuesta devuelve un resultado o un error con el mismo identificador. La especificación define dos transportes: *stdio*, en el que el cliente lanza el servidor como subproceso, y *Streamable HTTP*, en el que el servidor atiende a múltiples clientes por un único *endpoint* HTTP (**MCPSpec2025**). La revisión vigente hizo el protocolo sin estado: eliminó las sesiones y cada solicitud lleva su versión y sus capacidades (**MCPSpec2026**).

Un servidor ofrece tres primitivas: los recursos (*resources*), contexto y datos para el usuario o el modelo; los *prompts*, mensajes con plantilla y flujos de trabajo; y las herramientas (*tools*), funciones que el modelo ejecuta (**MCPSpec2026**). Ehtesham2025<empty citation> las clasifican según quién las controla: las herramientas por el modelo, los recursos por la aplicación y los *prompts* por el usuario. La herramienta es la primitiva central de este trabajo. Cada una declara nombre, descripción y un esquema de entrada en JSON Schema; el cliente las descubre con `tools/list`, las invoca con `tools/call` y recibe contenido no estructurado o estructurado, validable contra un esquema de salida (**MCPSpec2025**). La especificación separa los errores de protocolo de los errores de ejecución, como una regla de negocio violada, y exige que siempre exista una persona con capacidad de denegar una invocación. El cuadro 2.9 resume las primitivas y el uso que se prevé darles.

Cuadro 2.9. Primitivas del servidor MCP y su uso previsto en el proyecto

Primitiva	Control	Definición	Uso previsto
Herramientas (<i>tools</i>)	Modelo	Funciones que el modelo descubre e invoca, con esquema de entrada y salida	Consultas de lectura sobre el ERP (centros de costo, certificaciones, presupuesto) y sobre las predicciones del servicio FastAPI
Recursos (<i>resources</i>)	Aplicación	Contexto y datos que el host adjunta al modelo	No se contempla en el alcance; posible exposición futura de documentación del ERP
<i>Prompts</i>	Usuario	Mensajes con plantilla y flujos de trabajo	No se contempla; corresponden al asistente que queda como trabajo futuro

Fuente: Elaboración propia a partir de MCPSpec2026<empty citation> y Ehtesham2025<empty citation>.

El protocolo se apoya en la investigación sobre uso de herramientas por modelos de lenguaje. Schick2023<empty citation> parten de que los modelos fallan en tareas básicas, como la aritmética o la consulta de hechos, y entrenan uno capaz de decidir qué API llamar, cuándo, con qué argumentos y cómo incorporar el resultado. Qin2024<empty citation> llevan el problema a más de 16 000 API REST reales, y Patil2024<empty citation> identifican el modo de fallo característico: argumentos incorrectos y alucinación del uso de una API, que se mitiga cuando el modelo consulta documentación en tiempo de ejecución. Esos trabajos resuelven el lado del modelo; MCP estandariza el opuesto, cómo un servidor describe sus herramientas con esquemas explícitos y las ejecuta, de modo que cualquier host las descubra sin los adaptadores a medida que Ehtesham2025<empty citation> señalan como obstáculo entre marcos de agentes. Los mismos autores sitúan a MCP como el primer escalón, el acceso a herramientas, frente a los protocolos entre agentes, que quedan fuera del alcance de este trabajo.

2.9.2 INTEGRACIÓN DE SISTEMAS EMPRESARIALES CON MODELOS DE LENGUAJE

Un ERP acumula datos transaccionales estructurados, actualizados a diario y sujetos a permisos por usuario, y eso condiciona cómo ponerlos al alcance de un modelo de lenguaje. **Lewis2020**^{<empty citation>} señalan que los modelos preentrenados almacenan conocimiento factual en sus parámetros, pero acceden a él y lo manipulan con precisión limitada, y que dar procedencia a sus respuestas y actualizar su conocimiento siguen siendo problemas abiertos. Su propuesta, la generación aumentada por recuperación (*retrieval-augmented generation*, RAG), combina esa memoria paramétrica con un índice vectorial consultado por un recuperador neuronal y produce lenguaje más específico y factual. Ese patrón recupera pasajes por similitud. En un ERP, en cambio, la pregunta habitual (cuánto lleva certificado un centro de costo, qué probabilidad de sobre costo estima el modelo) se responde con una consulta estructurada sobre la fuente vigente y con los permisos de quien pregunta. **Anthropic2024MCP**^{<empty citation>} plantea que un desarrollador puede exponer sus datos mediante servidores MCP o construir las aplicaciones cliente que los consumen; este proyecto toma el primer rol, con lo que la lógica de negocio y el control de acceso permanecen en el ERP.

El servidor del proyecto corre en Docker como proceso independiente y lo consumen clientes remotos, por lo que el transporte que corresponde es Streamable HTTP. La especificación define su flujo de autorización para los transportes HTTP, mientras que para *stdio* recomienda tomar las credenciales del entorno (**MCPSpec2026**). En ese flujo el servidor MCP actúa como servidor de recursos OAuth 2.1, el cliente MCP como cliente OAuth 2.1 y el servidor de autorización emite los tokens. El servidor MCP debe implementar *OAuth 2.0 Protected Resource Metadata* (**RFC9728**): ante una solicitud sin credenciales responde 401 con un documento de metadatos que declara su servidor de autorización. El token viaja como *bearer* en cada solicitud, nunca en la URL, y el servidor debe validar que fue emitido específicamente para él como audiencia, conforme a *Resource Indicators for OAuth 2.0* (**RFC8707**); un token inválido recibe 401 y un alcance insuficiente, 403 (**MCPSpec2026**).

De ahí se desprende una consecuencia de diseño. Si el servidor MCP consulta API ascendentes, actúa ante ellas como cliente OAuth con un token separado y no debe reenviar el que recibió del cliente MCP, para evitar el problema del delegado confundido (**MCPSpec2026**). En

el proyecto, el token que valida el servidor MCP lleva como audiencia su URL canónica y no puede viajar al ERP en NestJS ni al servicio en FastAPI; hace falta un segundo token hacia esos servicios, cuyo mecanismo se define en los Capítulos III y IV. Del lado del proveedor de identidad, la guía oficial de **Keycloak2026MCP**<empty citation> documenta el uso de Keycloak como servidor de autorización para MCP y declara que no soporta *Resource Indicators*; la audiencia se fija con un *client scope* y un *Audience mapper* que incluye la URL del servidor MCP en el token, mientras el cliente MCP sigue enviando el parámetro `resource` porque la especificación lo exige aunque el servidor de autorización no lo procese. La verificación del token en sí se trata en la sección 2.8.

La autorización no agota la seguridad. **Hou2025**<empty citation> sistematizan las amenazas por fase del ciclo de vida del servidor y por tipo de atacante, entre ellas el robo de credenciales durante la invocación y los conflictos de nombres que llevan a ejecutar la herramienta equivocada o escalar privilegios. **Radosevich2025**<empty citation> demuestran que modelos de primera línea pueden ser inducidos a usar herramientas MCP para ejecutar código malicioso, tomar control remoto o robar credenciales, y proponen un auditor que genera casos adversarios a partir de lo que un servidor expone. La especificación reconoce que las herramientas representan ejecución de código arbitrario y establece que el servidor debe validar las entradas, controlar el acceso, limitar la tasa de invocaciones y sanear las salidas (**MCPSpec2025**). En este proyecto eso se traduce en un servidor que expone únicamente consultas de lectura acotadas sobre el ERP y las predicciones, sin nómina ni datos personales, con esquemas estrictos y acceso delegado en los alcances de Keycloak. Con ello el servidor deja el ERP y las predicciones listos para un asistente conversacional futuro; construir ese asistente, que es el host donde se materializan los ataques descritos, queda fuera del alcance.

2.10 INTELIGENCIA DE NEGOCIOS

2.10.1 DEFINICIÓN Y REPORTES DESCRIPTIVOS

La inteligencia de negocios (*business intelligence*, BI) designa el conjunto de tecnologías de apoyo a la decisión que permiten a ejecutivos, gerentes y analistas tomar decisiones mejores

y más rápidas (**Chaudhuri2011**). **Negash2004**<empty citation> la formula desde los sistemas de información: los sistemas de BI combinan datos operativos con herramientas analíticas para presentar información compleja a quienes planifican y deciden, con el fin de mejorar la oportunidad y la calidad de los insumos de la decisión. **Watson2007**<empty citation> la describen como un proceso con dos actividades principales: introducir los datos (*getting data in*), que corresponde al almacenamiento de datos tradicional y consiste en mover datos desde los sistemas fuente hacia un almacén integrado, y extraerlos (*getting data out*), cuando usuarios y aplicaciones acceden a ese almacén para reportes empresariales, consultas, procesamiento analítico y analítica predictiva. Los mismos autores advierten que la primera actividad aporta por sí sola un valor limitado: la organización solo obtiene el valor completo cuando usa los datos para decidir.

Esa arquitectura tiene componentes reconocibles. **Chaudhuri1997**<empty citation> presentan el almacén de datos y el procesamiento analítico en línea (*online analytical processing*, OLAP) como elementos esenciales del apoyo a la decisión y recorren sus piezas: herramientas de extracción, limpieza y carga (los procesos ETL de la sección 2.3), modelos de datos multidimensionales organizados en medidas y dimensiones, herramientas cliente de consulta y análisis, y gestión de metadatos. **Kimball2013**<empty citation> sistematizan el diseño de esa capa mediante el modelado dimensional: tablas de hechos que registran las medidas del negocio y tablas de dimensión que las describen, dispuestas en esquema en estrella. La aplicación de presentación con la que interactúa el usuario es el último eslabón de una cadena cuya calidad depende de las capas anteriores (**Chaudhuri2011**).

El producto más visible de la BI es el reporte descriptivo. **Lepeniotti2020**<empty citation> ordenan la analítica de negocios en tres niveles: la analítica descriptiva, que muestra lo que ocurrió; la predictiva, que estima lo que ocurrirá; y la prescriptiva, que busca el mejor curso de acción futuro y constituye, según los autores, el siguiente paso en la madurez analítica, mientras que el foco académico e industrial sigue puesto en las dos primeras. Los reportes descriptivos suelen entregarse como tableros (*dashboards*): **Eckerson2010**<empty citation> los caracteriza como una capa de presentación sobre la infraestructura de BI que integra el monitoreo, el análisis y la gestión de indicadores de desempeño, y **Few2013**<empty citation> aporta los criterios de diseño visual para que la información más importante pueda monitorearse de un vistazo en una sola pantalla. La BI de autoservicio, por último, aspira a que usuarios no especialistas exploren los

datos y deriven información accionable sin depender de los especialistas de BI (**Alpar2016**).

Estas nociones fijan el lugar de la BI en el proyecto. El pipeline de datos de la sección 2.3 es la actividad de introducir los datos; los reportes de Power BI y el tablero del ERP en Angular son dos canales para extraerlos. El marco evolutivo de **Chen2012<empty citation>**, que distingue etapas de la BI y la analítica según los datos que procesan, ubica a este trabajo en su punto de partida: datos estructurados de un sistema transaccional. La separación entre canales es una decisión de arquitectura y no una definición, puesto que **Watson2007<empty citation>** incluyen la analítica predictiva dentro de la BI. Con la tríada de **Lepenioti2020<empty citation>** como criterio, se prevé que Power BI cubra la analítica descriptiva del ERP (horas, compras, presupuesto ejecutado y certificaciones por centro de costo), que el módulo de aprendizaje automático cubra la predictiva dentro de la interfaz transaccional, y que la prescriptiva quede como trabajo futuro. Las razones técnicas de esa separación se desarrollan en la subsección siguiente.

2.10.2 POWER BI

Power BI es la plataforma de analítica de negocios de Microsoft, un conjunto integrado de herramientas para conectar, visualizar y compartir datos, con dos componentes principales: Power BI Desktop, aplicación gratuita para el modelado y la creación de reportes, y el servicio Power BI, orientado a compartir mediante espacios de trabajo y refresco programado (**MSPowerBIOverview**; **MSDAXOverview**). La preparación de los datos corre por cuenta de Power Query, un motor que realiza la extracción, transformación y carga (ETL) desde cientos de fuentes, registra cada transformación como un paso de consulta en el lenguaje M y la aplica sin modificar la fuente (**MSPowerQueryWhatIs**). El modelado se apoya en DAX (*Data Analysis Expressions*), lenguaje de fórmulas sobre tablas relacionadas de un modelo tabular con el que se definen medidas, columnas calculadas y funciones de inteligencia de tiempo (**MSDAXOverview**). El modelo semántico (antes *dataset*) es el contenedor sobre el que se construyen los reportes, y *dashboard* designa allí una pantalla de mosaicos (**MSPowerBIServiceConcepts**); en este documento se reserva “tablero” para el sentido de gestión de la subsección anterior.

El modo de acceso a la fuente condiciona el diseño. En el modo de importación los datos se

cargan en la caché en memoria del modelo semántico y los visuales no reflejan cambios en la fuente hasta el siguiente refresco; en DirectQuery cada visual consulta la fuente en el momento. Microsoft recomienda importar por defecto, y en su documentación las fuentes habilitadas para DirectQuery son bases de datos, mientras que los sitios web figuran entre las fuentes de importación (**MSPowerBIDirectQuery**). Importar implica refrescar: en capacidad compartida el servicio limita cada modelo a ocho refrescos programados por día (**MSPowerBIRefresh**). Si el servicio no puede establecer una conexión de red directa con la fuente, el refresco requiere una puerta de enlace de datos local (*on-premises data gateway*) instalada en la red de la empresa (**MSPowerBIRefresh**; **MSPowerBIGateway**); como la fase 1 corre en un servidor propio, su necesidad depende de si la API del ERP es alcanzable desde Internet, condición que el Capítulo III debe resolver. Estas propiedades fundamentan la decisión anterior: una copia importada que se actualiza a lo sumo ocho veces al día sirve para reportes descriptivos, no para predicciones calculadas bajo demanda sobre proyectos activos y atadas a la sesión del usuario.

El segundo condicionante es la autenticación. Power BI llega a la API de NestJS por el conector Web, cuya función `Web.Contents` admite, entre otros, los tipos Anonymous, Web API (clave de API) y Organizational Account, permite definir encabezados HTTP y envuelve una respuesta JSON con `Json.Document`; las solicitudes POST solo pueden hacerse de forma anónima (**MSPowerQueryWeb**). El flujo OAuth integrado de Organizational Account solo se inicia cuando el servicio responde 401 con un encabezado que remite a Microsoft Entra ID; más control sobre el flujo exige un conector personalizado (**MSPowerQueryAuth**), que puede declarar el tipo OAuth e implementar `StartLogin`, `FinishLogin` y `Refresh` con PKCE, o el tipo Key, cuya clave el motor inserta en el encabezado `Authorization` (**MSPowerQueryHandlingAuth**). Como el proveedor de identidad del proyecto es Keycloak y no Entra ID, quedan dos caminos abiertos: un conector personalizado con OAuth y PKCE contra Keycloak, o una credencial de servicio emitida por Keycloak cuyo token se pasa en el encabezado desde `Web.Contents`; la restricción sobre POST afecta a la segunda si el intercambio de credenciales se hiciera desde M. La elección se justifica en el Capítulo III y se construye en el incremento 5.

La elección frente a otras herramientas responde a un criterio técnico derivado del alcance: la BI debe leer el ERP por una API protegida, no por acceso directo a PostgreSQL. Apache Superset se conecta a bases de datos SQL (**ApacheSuperset**); Metabase documenta controladores para

bases de datos y no menciona API REST como fuente (**MetabaseDatabases**); Tableau accede a fuentes HTTP sin conector nativo mediante un *web data connector* en JavaScript, cuya versión 2.0 está obsoleta desde la versión 2023.1 (**TableauWDC**). Power BI trae de fábrica el conector Web con JSON y encabezados, y su herramienta de autoría es gratuita. El cuadro 2.10 resume la comparación; no se afirma una superioridad general, sino la adecuación a este caso. Con esa base, se prevé publicar el modelo semántico en el servicio Power BI y compartir los reportes con los jefes de proyecto y administración en la modalidad de autoservicio descrita antes.

Cuadro 2.10. Herramientas de BI consideradas y su forma de acceso a los datos del ERP

Herramienta	Fuentes documentadas	Acceso a la API del ERP
Power BI	Más de 100 fuentes: bases de datos, servicios en la nube, archivos y fuentes web	Conector Web de fábrica: JSON, encabezados HTTP, clave de API. OAuth integrado solo con Entra ID; otro proveedor exige conector personalizado
Apache Superset	Bases de datos SQL	Sin conector REST documentado; requeriría acceso directo a la base, excluido por el alcance
Metabase	Controladores para bases de datos (PostgreSQL, MySQL, SQL Server, entre otras)	Sin conector REST documentado; misma restricción
Tableau	Conectores nativos y <i>web data connector</i> para fuentes HTTP	Requiere programar un conector en JavaScript (WDC 3.0; la 2.0 está obsoleta)

Fuente: Elaboración propia a partir de **MSPowerBIOverview**<empty citation>, **MSPowerQueryAuth**<empty citation>, **ApacheSuperset**<empty citation>, **MetabaseDatabases**<empty citation> y **TableauWDC**<empty citation>.

CAPÍTULO III

ANÁLISIS Y DISEÑO DEL SISTEMA PROPUESTO

3.1 ANÁLISIS DE DATOS HISTÓRICOS

3.1.1 EXPLORACIÓN Y CARACTERIZACIÓN DE LAS FUENTES DE DATOS

3.1.2 IDENTIFICACIÓN DE VARIABLES PREDICTORAS

3.1.3 DEFINICIÓN DE CASOS DE USO PREDICTIVOS

3.2 ANÁLISIS DE PROCESOS DEL PORTAL ANTERIOR

3.2.1 RELEVAMIENTO DE PROCESOS DEL PORTAL ANTERIOR

3.2.2 PROCESOS A REIMPLEMENTAR EN EL ERP

3.2.3 DEFECTOS Y BRECHAS QUE AFECTAN LA CALIDAD DE LOS DATOS

3.3 REQUERIMIENTOS DEL SISTEMA

3.3.1 REQUERIMIENTOS FUNCIONALES

3.3.2 REQUERIMIENTOS NO FUNCIONALES

3.4 DISEÑO DE LA ARQUITECTURA

3.4.1 ARQUITECTURA POR CAPAS

3.4.2 DISEÑO DEL PIPELINE DE DATOS

3.4.3 DISEÑO DE MODELOS ML

3.4.4 DISEÑO DE LA API PREDICTIVA

3.4.5 DISEÑO DEL DASHBOARD

3.4.6 DISEÑO DE LA AUTENTICACIÓN CENTRALIZADA

3.4.7 DISEÑO DE LAS INTEGRACIONES CON MCP Y POWER BI

CAPÍTULO IV

CONSTRUCCIÓN E IMPLEMENTACIÓN DEL SISTEMA

4.1 INCREMENTO 1: PROCESOS DEL PORTAL ANTERIOR MIGRADOS AL ERP

4.2 INCREMENTO 2: AUTENTICACIÓN CENTRALIZADA CON KEYCLOAK

4.3 INCREMENTO 3: PIPELINE DE DATOS Y ENTRENAMIENTO DE MODELOS

4.3.1 INTEGRACIÓN DUCKDB CON MYSQL Y POSTGRESQL

4.3.2 TRANSFORMACIÓN DE DATOS CON POLARS Y FEATURE ENGINEERING

4.3.3 MODELO DE DESVIACIÓN EN CERTIFICACIONES

4.3.4 MODELO DE SOBRECOSTO DE PROYECTOS

4.3.5 MODELO DE RETRASO EN LA FECHA DE CIERRE

4.4 INCREMENTO 4: MICROSERVICIO FASTAPI E INTEGRACIÓN CON EL ERP

4.4.1 DESARROLLO DEL MICROSERVICIO FASTAPI

4.4.2 ENDPOINTS DE PREDICCIÓN EN NESTJS

4.4.3 DASHBOARD DE INTELIGENCIA PREDICTIVA EN ANGULAR

4.5 INCREMENTO 5: INTEGRACIÓN CON POWER BI Y MCP

4.5.1 API DE DATOS PARA POWER BI

4.5.2 SERVIDOR MCP

4.6 EVALUACIÓN Y VALIDACIÓN

4.6.1 MÉTRICAS DE RENDIMIENTO DE MODELOS

4.6.2 PRUEBAS CON USUARIOS CLAVE

CONCLUSIONES

RECOMENDACIONES

REFERENCIAS BIBLIOGRÁFICAS

- Al Jafari, A., & Nair, S. S. K. (2018). ERP Implementation in the Oil and Gas Sector: A Case Study in Sultanate of Oman. *2018 7th International Conference on Reliability, Infocom Technologies and Optimization (Trends and Future Directions) (ICRITO)*, 848-854. <https://doi.org/10.1109/ICRITO.2018.8748779>
- Databricks. (2025). *What is the medallion lakehouse architecture?* Databricks Documentation. Consultado el 19 de junio de 2026, desde <https://docs.databricks.com/aws/en/lakehouse/medallion>
- Habibi, M., Kermanshachi, S., & Safapour, E. (2018). Engineering, Procurement, and Construction Cost and Schedule Performance Leading Indicators: State-of-the-Art Review. *Construction Research Congress 2018*, 378-388. <https://doi.org/10.1061/9780784481271.037>
- Inan, T., Narbaev, T., & Hazir, Ö. (2022). A Machine Learning Study to Enhance Project Cost Forecasting. *IFAC-PapersOnLine*, 55(10), 3286-3291. <https://doi.org/10.1016/j.ifacol.2022.10.127>
- Jawad, Z. N., & Villányi, B. (2024). Machine learning-driven optimization of enterprise resource planning (ERP) systems: a comprehensive review. *Beni-Suef University Journal of Basic and Applied Sciences*, 13(1), Artículo 4. <https://doi.org/10.1186/s43088-023-00460-y>
- Mohd Roshdi, F. R., Ismail, K., Lop, N. S., & Ab Wahab, L. (2022). Cost Overruns in Engineering Procurement Construction (EPC) Fabrication Oil and Gas Megaprojects in Malaysia: The Importance of Resource Allocation (5M). *International Journal of Academic Research in Business and Social Sciences*, 12(9). <https://doi.org/10.6007/ijarbss/v12-i9/14752>
- Nguyen, M.-T., Phan, V.-T., Tran, H.-L., & Nguyen Vu, M.-A. (2026). An explainable XGBoost model for predicting cost overruns in construction projects. *Asian Journal of Civil Engineering*, 27(7), 3611-3628. <https://doi.org/10.1007/s42107-026-01686-8>
- Ottaviani, F. M., Ballesteros-Pérez, P., & Narbaev, T. (2025). Automated machine learning pipeline for robust project cost and duration forecasting. *Automation in Construction*, 178. <https://doi.org/10.1016/j.autcon.2025.106426>
- Reis, J., & Housley, M. (2022). *Fundamentals of Data Engineering: Plan and Build Robust Data Systems*. O'Reilly Media.
- Shamim, M. M. I., Abdul Hamid, A. B. b., Nyamasvisva, T. E., & Rafi, N. S. B. (2025). Advancement of Artificial Intelligence in Cost Estimation for Project Management Success: A Systematic Review of Machine Learning, Deep Learning, Regression, and Hybrid Models. *Modelling*, 6(2), 35. <https://doi.org/10.3390/modelling6020035>
- Shen, Z., Wang, X., Zhang, Z., & Kim, H. (2025). Domain-Aware Machine Learning for Project Delay Prediction: A Rule-Constrained XGBoost Framework. *2025 10th International Conference on Computer and Information Processing Technology (ISCIPT)*, 9-13. <https://doi.org/10.1109/ISCIPT67144.2025.11265281>